SOFTWARE ENGINEERING PROJECT

Vid-Scratch
(Project Proposal)

BY

Nantawan Paramapooti
Pichayoot Tanasinanan

DEPARTMENT OF COMPUTER ENGINEERING
FACULTY OF ENGINEERING
KASETSART UNIVERSITY

Academic Year 2026

Vid-Scratch
(Project Proposal)

BY

Nantawan Paramapooti
Pichayoot Tanasinanan

This Project Submitted in Partial Fulfillment of the Requirement for Bachelor Degree of Engineering (Software Engineering)
Department of Computer Engineering, Faculty of Engineering KASERTSART UNIVERSITY
Academic Year 2026

Approved By:

Advisor . . . . . **Date** . . . . . .
(Punpiti Piamsa-nga)

Co-Advisor . . . . . **Date** . . . . . .
(Prof. Chantana Chantrapornchai)

Head of Department . . . . . **Date** . . . . . .
(Punpiti Piamsa-nga)

Abstract

Video-based action recognition models are widely studied in action recognition research and are increasingly relevant in applications that require reliable understanding of human actions from video. As these models become more important, understanding their susceptibility to adversarial manipulation becomes a meaningful problem in machine learning security. This project presents Vid-Scratch, an adversarial video poisoning pipeline developed to investigate the vulnerability of action recognition models under two threat scenarios: an inference-time attack, in which an adversarially perturbed video causes a pre-trained model to misclassify an action despite appearing visually similar to the original clip, and a training-time attack, in which poisoned video samples are introduced into the training set to degrade the performance of a newly trained model.

The proposed system targets SlowFast R101, a dual-pathway action recognition architecture pretrained on Kinetics-400, and combines Bayesian Optimization for keyframe selection, spatial transformation through a learned flow field, and additive adversarial noise optimized using gradient-based methods. The perturbations are constrained by both a pixel-level noise budget and a perceptual similarity measure based on SSIM [1]. For the training-time setting, the pipeline also incorporates an Error-Minimizing Noise component intended to reduce the usefulness of poisoned samples during model learning.

Vid-Scratch is evaluated on SlowFast R101 using video samples from the Kinetics-400 action recognition dataset. The evaluation focuses on attack success rate in the inference-time setting, as well as classification accuracy degradation, loss behavior, and perceptual quality in the training-time setting. Through this evaluation, the project aims to assess whether adversarial video perturbations can effectively compromise a strong temporal action recognition model while preserving high visual similarity to the original content.

Keywords: Video Poisoning, Adversarial Attack, Action Recognition, SlowFast, Data Poisoning, AI Security

Acknowledgement

We would like to express our sincere gratitude to the Department of Computer Engineering, Kasetsart University for providing us with invaluable resources, technical knowledge, and support throughout the development of this project. Their guidance has been instrumental in shaping the Vid Scratch system.

We are deeply thankful to our advisor, Assoc. Prof. Dr. Punpiti Piamsa-nga, for his expertise and mentorship in the field of image processing and adversarial attacks, which have been critical to the project's success. We also extend our heartfelt appreciation to Prof. Chantana Chantrapornchai, Ph.D., for her insightful guidance on parallel computing and optimization, enabling us to refine the efficiency of our AI pipeline.

Additionally, we would like to acknowledge the research communities and open-source contributors whose work on adversarial robustness, video processing, and AI security provided essential knowledge and tools that contributed to this project's implementation.

Finally, we would like to thank our faculty members, colleagues, and everyone who has supported and encouraged us throughout this journey. Their insights, discussions, and feedback have been invaluable in developing a meaningful and impactful solution to counter AI exploitation in video generation.

Nantawan Paramapooti
Pichayoot Tanasinanan

Table of Contents

List of Tables

Page

List of Figures

Chapter 1

Introduction

1.1 Background

Video-based action recognition[0] is a class of deep learning[1] systems that identifies human actions from video clips by analyzing both the visual appearance of individual frames and the motion patterns across frames over time. Among the architectures[2] widely studied in this area, the Inflated 3D ConvNet (I3D)[3] is one of the most recognized earlier models [2]. By extending conventional 2D convolutional networks into the temporal domain[4] through 3D convolutions[5], I3D can jointly learn spatial[6] and temporal[7] features[8] from video data, and has become a standard reference model in action recognition research due to its strong performance on benchmark datasets[9] such as Kinetics and UCF-101. Building on this line of work, SlowFast R101[3b] extends the approach by processing video through two parallel pathways [3] operating at different temporal resolutions — a slow pathway that captures spatial semantics at low frame rate, and a fast pathway that captures motion at high frame rate — achieving stronger recognition performance on the same benchmarks. Due to these properties, SlowFast R101 represents a strong and well-studied target for evaluating adversarial robustness in video action recognition.

As video understanding models become more widely used, concerns about how reliably they behave under manipulated inputs have become increasingly important. In practice, video clips are no longer interpreted only by humans, but also by AI[10] systems that classify activities, summarize content, and support automated decision-making. For everyday users such as video creators, this raises a practical issue: once a video is uploaded or shared online, the content may be analyzed, categorized, or

learned by machine learning systems in ways the original owner cannot easily observe or control. Because of this, there is growing interest in tools that can help users better understand or influence how AI systems interpret their video content.

Adversarial attacks[11] provide one way to study this problem. These attacks introduce small, carefully designed perturbations[12] into input data [4] so that a model produces incorrect outputs while the modified content remains visually similar to the original for human viewers. Although adversarial attacks have been studied extensively for image classifiers, applying the same idea to video models is more challenging. Unlike image classification models that process one image at a time, video action recognition models process multiple frames jointly, integrating spatial and temporal information across the full clip. As a result, perturbations applied independently to individual frames may become less effective after temporal aggregation[13]. This makes video-based adversarial manipulation a more complex problem and highlights the need for attack methods that explicitly account for temporal structure[14].

These challenges motivate the development of Vid-Scratch. Rather than claiming to completely prevent AI training or fully protect video content from all forms of machine learning use, Vid-Scratch is designed as a user-oriented application for generating visually similar adversarial video variants and evaluating how susceptible temporal action recognition models are to such inputs. The system targets SlowFast R101, a strong dual-pathway model pretrained on Kinetics-400, and focuses on two threat scenarios[15]. In the first, a pre-trained[16] model receives an adversarially perturbed video at inference time[17] and misclassifies the action shown in the clip. In the second, poisoned video samples[18] are introduced into the training dataset, causing a model trained on that data to learn distorted patterns and achieve lower classification performance during evaluation. Through these two scenarios, the project aims to examine whether strong benchmark performance in action recognition also implies robustness[19] under adversarially manipulated video inputs.

In addition to its research value, the project also has practical relevance. For non-expert users, Vid-Scratch can be viewed as a tool for producing video versions that remain close to the original in human per-

ception while making AI-based action interpretation less reliable. For students, researchers, and developers, it serves as an accessible robustness-testing application that demonstrates how vulnerable temporal video models may be when exposed to carefully constructed perturbations. In this way, Vid-Scratch bridges technical adversarial machine learning research with a more user-friendly application setting.

1.2 Problem Statement

Although adversarial attacks have been widely studied in image classification, applying them effectively to video-based action recognition remains more difficult. Models such as SlowFast R101 process multiple frames jointly across two temporal pathways and rely on both spatial and temporal information to recognize actions. Because of this, perturbations designed independently for single images may become less effective when extended directly to video, especially after temporal information is aggregated across the full clip. This creates a practical challenge for studying how vulnerable video action recognition models are under adversarial manipulation.

At the same time, there is a lack of accessible tools that allow users to generate and evaluate adversarial video inputs against temporal action recognition models in a simple and systematic way. Existing adversarial workflows are often implemented as research code, require significant technical knowledge, and are not designed as user-oriented applications. For non-expert users, students, or developers who want to test the robustness of video models, building such an adversarial pipeline from scratch can be difficult and time-consuming.

This project addresses these problems by developing Vid-Scratch, a user-oriented adversarial video generation application targeting SlowFast R101. The project focuses on two main threat scenarios:

1. **Inference-time attack[20]:** determining whether an adversarially perturbed video can cause a pre-trained SlowFast R101 model to

misclassify the action shown in the clip while preserving high visual similarity to the original video.

2. **Training-time attack[21]:** determining whether poisoned video samples inserted into a training dataset can degrade the performance of a model trained on that data.

In addition, the project also addresses a usability problem by providing a more practical pipeline that enables users to generate adversarial video samples and evaluate model vulnerability without having to manually combine multiple low-level attack steps on their own.

Therefore, the core problem addressed in this project is not only whether SlowFast R101 is vulnerable to adversarial video manipulation, but also how such attacks can be generated and evaluated through an accessible application that bridges adversarial machine learning research and practical video robustness testing.

1.3 Solution Overview

Vid-Scratch is designed as a user-oriented application that simplifies the generation of adversarial video inputs[22] for evaluating the robustness of video-based action recognition models, particularly SlowFast R101. Instead of requiring users to manually perform multiple research-level processing steps, the software provides a more accessible workflow in which the user selects an input video, adjusts the relevant settings, starts the generation process, and receives the resulting adversarial or poisoned video output in a designated folder.

The system supports two main purposes. First, it can generate adversarially perturbed videos for inference-time evaluation, allowing users to test whether a pre-trained SlowFast R101 model will misclassify the action shown in the input video. Second, it can prepare poisoned video samples for training-time evaluation, allowing users to study whether modified training data can reduce the performance of a newly trained model. In both cases, the system is designed to preserve high visual similarity between the modified video and the original clip while still influencing model behavior.

Because SlowFast R101 processes only a limited number of frames from each input clip, the current scope of the application focuses on short videos. In this project, the system is designed for videos of no longer than one minute, which is sufficient for extracting the 40-frame representation used in the model pipeline while keeping processing practical and consistent with the project scope.

In addition to its value for adversarial machine learning experiments, Vid-Scratch also has practical relevance for non-expert users. A user may create an alternative version of a video that remains visually close to the original while making AI-based action recognition less reliable. The application can also be used as a simple robustness-checking tool, helping users examine whether their videos are easily recognized or categorized by an action recognition model[23].

Rather than claiming to completely prevent AI training or fully block all forms of machine learning use, Vid-Scratch is intended as a practical application for adversarial video generation, robustness testing, and AI-aware video transformation. In this way, the software bridges adversarial machine learning research with a more accessible application setting for students, researchers, developers, and everyday users.

1.3.1 Features

1. **Inference-Time Adversarial Video Generation:** Generate visually similar adversarial video variants from an input clip in order to influence the prediction behavior of a pre-trained SlowFast R101 model and evaluate whether the model misclassifies the action shown in the video.

2. **Training-Time Poisoned Sample Generation:** Produce poisoned video samples that can be inserted into a training dataset to evaluate whether modified training data degrades the performance of a newly trained model.

3. **Parameter Configuration:** Allow users to adjust relevant generation settings such as perturbation strength, optimiza-

tion parameters[24], and output quality according to the available attack pipeline configuration.

4. **Simple User Workflow:** Provide an accessible process in which the user selects a video, chooses an output location, configures settings, and runs the adversarial generation pipeline without manually combining multiple low-level attack steps.
5. **Model Robustness Evaluation Support[25]:** Support the evaluation of SlowFast R101 vulnerability by enabling users to examine whether a video can be misclassified at inference time or whether poisoned samples can reduce model performance during training.
6. **Short Video Support:** Support short video clips of up to one minute in length, which fit the current SlowFast R101-based processing pipeline that operates on a 40-frame representation of the input video.
7. **Video Format Support:** Support MP4 video input and output for the current version of the project, with the possibility of extending file format support in future development.
8. **Hardware-Aware Processing:** Use the available hardware resources to run the generation pipeline as efficiently as possible in order to reduce processing time for short video clips.

1.4 Target User

- **General Users and Digital Content Creators:** Individuals who share short videos online, including everyday users and creators of dance clips, exercise videos, sports demonstrations, tutorials, and other action-centered content. These users may want more control over how AI systems interpret their videos, especially when they do not want their content to be easily recognized, categorized, or

analyzed by action recognition models. The application allows them to generate visually similar video variants that may be more difficult for such models to interpret reliably.

- **Students, Researchers, and Developers:** Users who want a practical and accessible application for generating adversarial or poisoned videos in order to study the robustness of SlowFast R101 under inference-time and training-time attack scenarios. This group also includes users who want to demonstrate or explain how visually similar videos can still cause incorrect predictions in temporal action recognition models.
- **Machine Learning Practitioners and Evaluators:** Developers who need to test whether their action recognition models remain robust when exposed to adversarially perturbed or poisoned video inputs, and who may use the application as part of a robustness evaluation workflow.

1.5 Benefit

Vid-Scratch provides value in two main ways. From a user perspective, it gives people a practical way to create video variants that remain visually close to the original while reducing the reliability of AI-based action recognition. From a technical perspective, it offers an accessible application for evaluating the adversarial robustness of SlowFast R101 under both inference-time and training-time attack scenarios.

- Supports AI-aware video transformation for users who want more control over how their videos may be interpreted by action recognition models.
- Enables inference-time testing to observe whether adversarially perturbed videos can cause SlowFast R101 to misclassify actions.
- Enables training-time evaluation by generating poisoned samples that can be used to study performance degradation after training.

- Makes adversarial video experimentation more accessible by reducing the need for manual, low-level processing steps.

1.6 Timeline

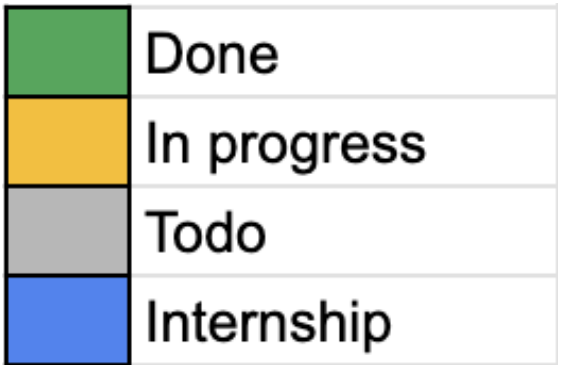

Figure 1.1: Color Meaning chart

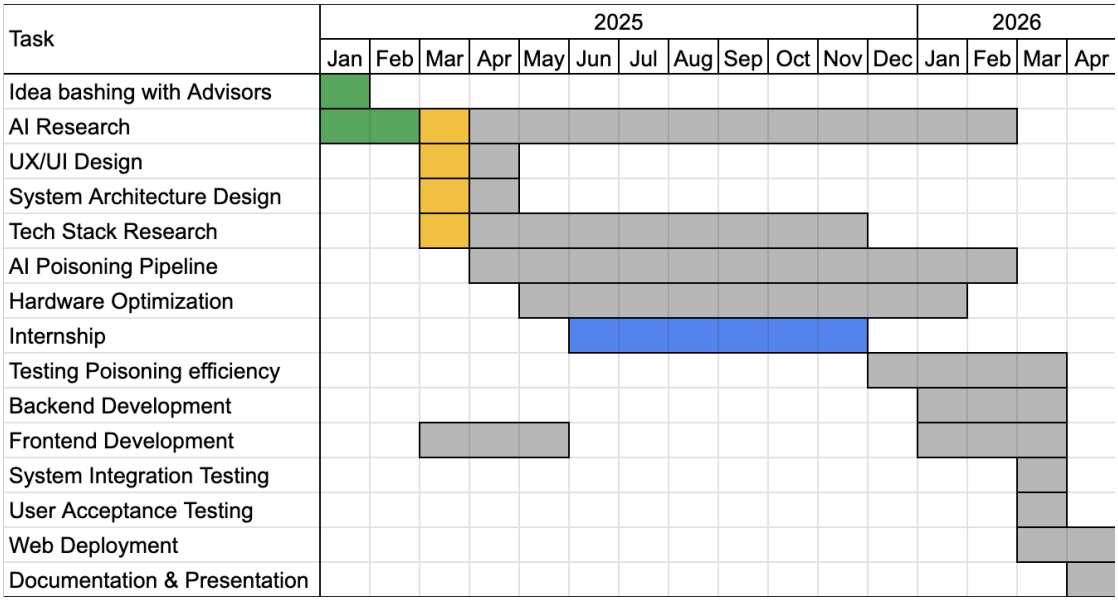

Figure 1.2: Timeline of Project development

In Figure 1.2. is the current timeline of our project plan. You may look up to Figure 1.1. for color code meanings.

1.7 Terminology

[0] Video-based action recognition : a type of AI system that identifies human actions from video clips.
[1] Deep learning : a branch of machine learning that uses neural networks with many layers to learn patterns from data.
[2] Architecture : the structural design of a neural network model.
[3] Inflated 3D ConvNet (I3D) : an earlier video action recognition model that extends 2D convolutional networks into 3D to process both space and time, and serves as a predecessor to more recent architectures such as SlowFast.
[3b] SlowFast R101 : a video action recognition model that processes video through two parallel pathways at different temporal resolutions — a slow pathway for spatial semantics and a fast pathway for motion — pretrained on Kinetics-400 and used as the target model in this project.
[4] Temporal domain : the time-related dimension of data, which captures changes across frames or moments.
[5] 3D convolutions : convolution operations applied across height, width, and time in video data.
[6] Spatial : related to the visual appearance or arrangement within a single frame.
[7] Temporal : related to time and changes across multiple frames.
[8] Features : meaningful patterns or representations learned by a model from input data.
[9] Benchmark datasets : standard datasets used to evaluate and compare model performance.
[10] AI : artificial intelligence.
[11] Adversarial attacks : methods that intentionally modify input data to cause a machine learning model to make incorrect predictions.
[12] Perturbations : small modifications added to input data, often designed to be difficult for humans to notice.
[13] Temporal aggregation : the process of combining information across multiple frames over time.
[14] Temporal structure : the arrangement and relationship of information across time in a video sequence.

[15] Threat scenarios : different situations or settings in which an attack may occur.
[16] Pre-trained : a model that has already been trained on a dataset before being used for another task or evaluation.
[17] Inference time : the stage when a trained model is used to make predictions on new input data.
[18] Poisoned video samples : video data that has been intentionally modified before training in order to influence how a model learns.
[19] Robustness : the ability of a model to maintain reliable performance when facing noise, perturbations, or adversarial inputs.
[20] Inference-time attack : an attack performed when a trained model is making predictions on new input data.
[21] Training-time attack : an attack that modifies training data in order to influence how a model learns.
[22] Adversarial video generation : the process of creating modified video inputs designed to mislead a machine learning model.
[23] Action recognition model : a model that identifies or classifies human actions from video data.
[24] Optimization parameters : configurable values that control how an optimization process updates or generates perturbations.
[25] Robustness evaluation : the process of testing how reliably a model performs under perturbed, noisy, or adversarial inputs.

Chapter 2

Literature Review and Related Work

Video-based action recognition has become an important research area in computer vision, with many studies focusing on how deep learning models can recognize human actions from video clips by learning both spatial and temporal information. Among the most influential architectures in this domain is the Inflated 3D ConvNet (I3D), which extends 2D convolutional networks into the temporal dimension through 3D convolutions and has demonstrated strong performance on benchmark datasets such as Kinetics and UCF-101. Because of its strong recognition capability and representative role in video understanding research, I3D is a suitable model for studying adversarial vulnerability in action recognition systems.

At the same time, adversarial machine learning has shown that deep neural networks [5] can be misled by carefully designed perturbations that remain difficult for humans to notice. While this problem has been studied extensively in image classification, transferring the same ideas to video models is more challenging because video-based models rely not only on visual appearance within each frame, but also on temporal relationships across multiple frames. As a result, adversarial attacks on video action recognition require methods that consider temporal structure rather than applying image-based perturbations independently frame by frame.

Therefore, this chapter reviews prior work in three areas relevant to Vid-Scratch: video-based action recognition models, adversarial attacks on deep learning systems, and research on adversarial manipulation in video understanding. It also examines the gap between existing research-oriented attack methods and the need for a more accessible application that can be used to generate and evaluate adversarial video inputs for I3D under both inference-time and training-time scenarios.

2.1 Competitor Analysis

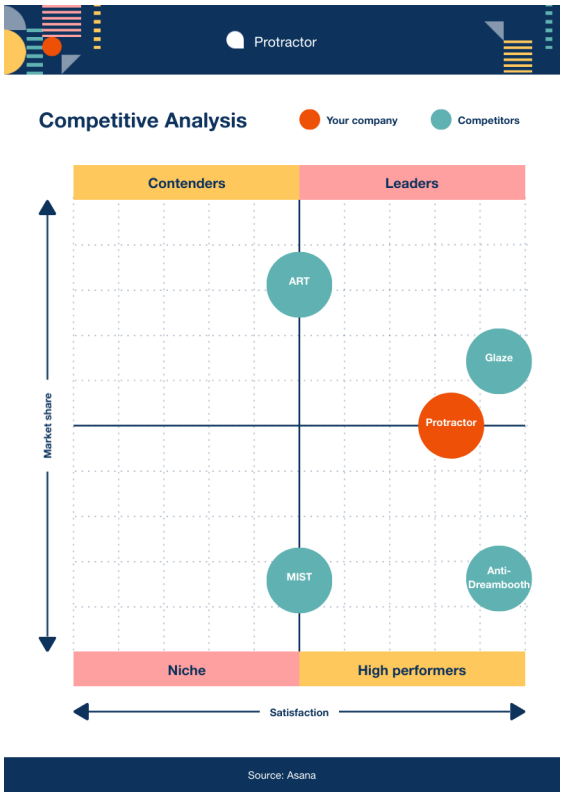

Figure 2.1: Competitive Landscape

Existing work related to Vid-Scratch can be divided into two groups. The first group consists of research code specifically designed for adversarial attacks on video-based action recognition or temporal video models. These works are the closest to the scope of this project because they directly address adversarial manipulation in video understanding. The second group consists of general adversarial machine learning frameworks that provide useful attack implementations, but are mainly designed for images and do not directly support adversarial video generation for temporal action recognition models such as I3D.

The main gap identified from these existing works is that they are largely research-level implementations. They are typically designed for experimentation, paper reproduction, or technical evaluation, and are not

presented as accessible user-oriented applications. Vid-Scratch addresses this gap by transforming adversarial video generation and evaluation into a more practical and usable application setting.

1. **DeepSAVA**

   DeepSAVA is the most closely related work to this project because it directly targets adversarial attacks on video-based action recognition and serves as the main technical foundation for the implementation of Vid-Scratch. It provides a research-oriented method for generating adversarial video perturbations against temporal models, making it highly relevant to the inference-time and training-time attack scenarios studied in this project. However, DeepSAVA is presented as research code rather than a user-facing application, which limits its accessibility for non-expert users.

2. **BTC-UAP**

   BTC-UAP (Breaking Temporal Consistency with Universal Adversarial Perturbations) is another important reference in adversarial video research. Its main idea is to exploit temporal inconsistency in order to reduce the reliability of video model predictions. This makes it relevant to action recognition tasks where temporal information is central to classification. However, like DeepSAVA, BTC-UAP is primarily a research implementation and does not provide an accessible workflow for general users who want to generate or evaluate adversarial video through an application interface.

3. **stAdv**

   stAdv (Spatially Transformed Adversarial Attack) is a spatial transformation-based adversarial attack that generates perturbations by warping image structure rather than relying only on additive noise. Its ideas are relevant to this project because spatial transformation is one of the components incorporated into the Vid-Scratch pipeline. Nevertheless, stAdv is still a research-focused attack method and is not designed as a complete user-oriented application for temporal video attack generation.

4. **ART : Adversarial Robustness Toolbox**

The Adversarial Robustness Toolbox (ART) is a widely used framework for adversarial machine learning research. It provides many attack and defense methods and is useful for experimentation in image-based settings. However, its support is primarily centered on image and general machine learning workflows, and it does not directly provide the kind of video-focused adversarial generation pipeline needed for temporal action recognition models such as I3D.

5. **Foolbox**

   Foolbox is another widely used adversarial attack framework that offers standardized implementations of many attack algorithms. Similar to ART, it is highly useful for adversarial experimentation in image classification and related tasks, but it is not specifically designed for adversarial video generation or for user-oriented robustness testing on temporal action recognition models.

In summary, DeepSAVA, BTC-UAP, and stAdv are the most relevant prior works from the perspective of adversarial video research, but all three are research-level implementations rather than accessible applications. ART and Foolbox provide strong general-purpose adversarial attack frameworks, but they do not directly support the video-based, I3D-focused workflow required in this project. Therefore, the main contribution of Vid-Scratch is not only the use of adversarial video attack methods, but also the presentation of these ideas through a more accessible application for robustness testing and AI-aware video transformation.

2.2 Literature Review

1. **stAdv**

   stAdv, or Spatially Transformed Adversarial Attack, is an adversarial attack method based on local geometric transformation rather than relying only on additive noise. Its main idea is to generate adversarial examples by slightly warping image structure [6] in a way that remains visually similar to the original input while still misleading the model. This work is relevant to Vid-Scratch because spatial transformation is

one of the important ideas incorporated into the project pipeline. In particular, it provides a useful foundation for designing perturbations that affect model prediction while preserving perceptual similarity between the original and modified output.

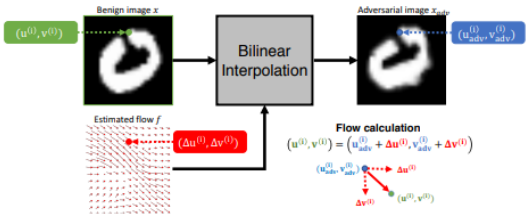

Figure 2.2: Explanation of how perturbation is added

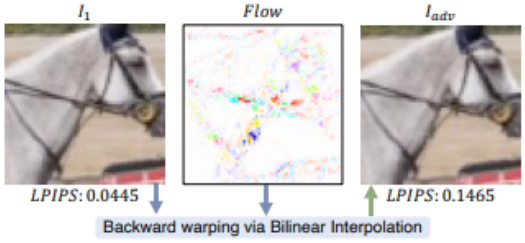

Figure 2.3: Explanation of how perturbation is added with LPIPS to measure perceptual similarity between the original and the poisoned output

Previous results in the literature show that stAdv can alter model attention while maintaining strong perceptual similarity, demonstrating that spatially transformed perturbations can be effective adversarial mechanisms. This makes stAdv an important reference for the spatial component of Vid-Scratch.

2. **BTC-UAP**

BTC-UAP, or Breaking Temporal Consistency with Universal Adversarial Perturbation, is a video-focused adversarial method that targets the temporal dimension of video models. Instead of treating each frame independently, BTC-UAP is designed to disturb temporal consistency across frames so that the model becomes less reliable when processing the full sequence [7]. This idea is especially relevant to

action recognition systems such as I3D, which rely on both spatial and temporal information to classify actions.

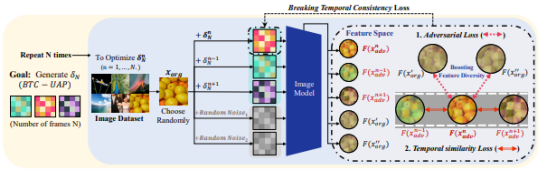

Figure 2.4: BTC-UAP perturbation logic

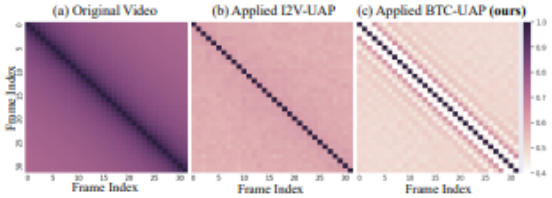

Figure 3: **The feature similarity of frames within videos.** This heatmap shows the average feature similarity between frames in the UCF-101 dataset, with brighter colors indicating lower levels of similarity.

Figure 2.5: Heatmap of temporal consistency; darker means better consistency

BTC-UAP is relevant to this project because it shows that adversarial attacks on video models must account for temporal relationships across frames rather than applying frame-level perturbations independently. This provides an important conceptual foundation for adversarial video generation against temporal action recognition models.

3. **DeepSAVA**

   DeepSAVA is the most directly relevant prior work for this project because it specifically targets adversarial attacks on video-based action recognition and serves as the main technical foundation of the VidScratch implementation [8]. It studies how adversarial perturbations can be generated for temporal video models and is closely aligned with the inference-time and training-time attack scenarios considered in this project. Compared with more general adversarial methods,

DeepSAVA is especially important because it addresses the practical difficulty of attacking video models that depend on both spatial and temporal reasoning.

DeepSAVA is therefore not only related work, but also a core reference for the design of Vid-Scratch. However, like many research papers in this area, it is presented primarily as a research-oriented implementation rather than as a user-facing application. This creates an opportunity for Vid-Scratch to build upon its technical ideas while presenting them in a more accessible and application-oriented form.

These studies provide the main technical foundations for Vid-Scratch. In particular, stAdv contributes ideas related to spatial transformation, BTC-UAP highlights the importance of disrupting temporal consistency, and DeepSAVA provides the most direct reference for adversarial attacks on video-based action recognition. Therefore, this project builds on these works to develop a more accessible application for generating and evaluating adversarial video inputs for I3D under both inference-time and training-time scenarios.

Chapter 3

Requirement Analysis

3.1 Stakeholder Analysis

1. **General Users and Digital Content Creators:** These stakeholders use Vid-Scratch to generate visually similar alternative versions of short videos while reducing the reliability of AI-based action recognition. They are concerned with having more control over how their videos may be interpreted, categorized, or analyzed by temporal video models.
2. **Students, Researchers, and Educators:** These stakeholders use the application as a practical tool for studying adversarial machine learning, demonstrating the vulnerability of temporal action recognition models, and experimenting with inference-time and training-time attack scenarios in an accessible way.
3. **Developers and Machine Learning Practitioners:** These stakeholders are interested in evaluating whether action recognition models such as I3D remain reliable when exposed to adversarially perturbed or poisoned video inputs. For this group, Vid-Scratch can support robustness testing and model evaluation workflows.
4. **Project Developers and Academic Supervisors:** These stakeholders are involved in the design, implementation, and evaluation of the system. They are responsible for ensuring that the project is technically sound, aligned with its academic objectives, and usable within the defined project scope.

3.2 User Stories

3.2.1 Generate AI-Aware Video Variants

As a general user or digital content creator,
I want to generate a visually similar version of my video that may be more difficult for action recognition models to interpret reliably,
so that I can have more control over how AI systems may analyze or categorize my video content.

3.2.2 Preserve Visual Similarity

As a general user, creator, or researcher,
I want to apply adversarial video generation while preserving the visual appearance of the original clip,
so that the modified video remains close to the original for human viewers.

3.2.3 Configure Generation Parameters

As a user of the Vid-Scratch application,
I want to adjust settings such as perturbation strength, optimization parameters, and output quality before running the process,
so that I can balance adversarial effectiveness, visual similarity, and processing cost according to my needs.

3.2.4 Test Inference-Time Vulnerability

As a student, researcher, or developer,
I want to generate adversarial video samples for inference-time testing,
so that I can examine whether a pre-trained I3D model misclassifies the action shown in the input video.

3.2.5 Prepare Training-Time Poisoned Samples

As a student, researcher, or machine learning practitioner,
I want to generate poisoned video samples for training-time experiments,
so that I can study whether modified training data reduces model performance after training.

3.2.6 Process Short Videos Efficiently

As a user working with short video clips,
I want to process videos of up to one minute in length within the application,
so that I can use the system within the scope supported by the I3D-based pipeline.

3.2.7 Monitor Generation Progress

As a user waiting for processing to complete,
I want to see the current status of adversarial video generation,
so that I can understand whether the process is running normally and when the output may be ready.

3.3 Use Case Diagram

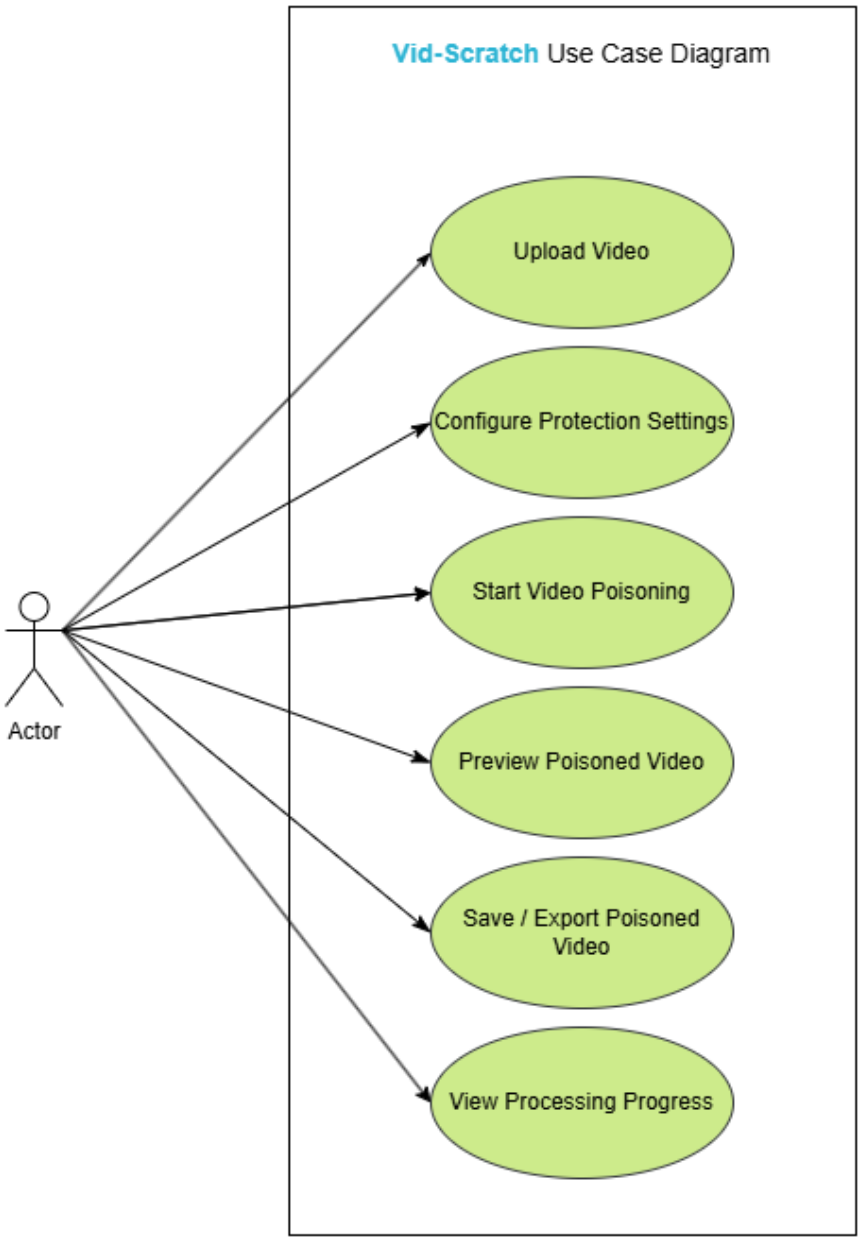

Figure 3.1: Use Case Diagram

The use case diagram above illustrates the main interactions between the user and the Vid-Scratch system. A single actor is used to

represent all user groups identified in the stakeholder analysis because the application provides the same core workflow to all users.

- ***User*** — the general actor representing any person who uses the Vid-Scratch application

The actor can perform six main use cases: uploading an input video, configuring generation settings, starting the adversarial generation process, previewing the output video, exporting the generated output to a designated location, and monitoring processing progress during generation.

3.4 Use Case Model

3.4.1 Use Case 1: Upload Video

Primary Actor: User
Description: The user uploads a short video file to the system in preparation for adversarial generation.
Scenario:

1. The user opens the Vid-Scratch application.
2. The system displays the upload interface.
3. The user selects a supported MP4 video file for upload.
4. The system validates the file format and duration, then stores it for processing.

Alternative Flow: If the file type is unsupported or the video exceeds one minute in length, the system displays an error message.

3.4.2 Use Case 2: Configure Generation Parameters

Primary Actor: User
Description: The user adjusts the adversarial generation settings before starting the process.
Scenario:

1. The user navigates to the settings panel after uploading a video.
2. The system displays available generation parameters, such as perturbation strength, optimization settings, and output quality.
3. The user adjusts the settings according to their needs.
4. The system saves the selected configuration for use in the generation pipeline.

3.4.3 Use Case 3: Start Adversarial Generation

Primary Actor: User
Description: After configuring the parameters, the user starts the adversarial video generation process.
Scenario:

1. The user selects the output folder and clicks the Start button.
2. The system begins the adversarial generation pipeline.
3. The system displays a progress indicator showing the current processing status.

Alternative Flow: If no video has been uploaded or no output folder has been selected, the system displays a warning message and does not start the process.

3.4.4 Use Case 4: Monitor Processing Progress

Primary Actor: User
Description: The user monitors the status of the adversarial generation process while it is running.
Scenario:

1. The system updates the progress indicator as processing continues.

2. The user monitors the current status of the pipeline.

3. Once complete, the system notifies the user that the output video is ready.

3.4.5 Use Case 5: Preview Output Video

Primary Actor: User

Description: The user previews the generated output video to verify the result before exporting it.

Scenario:

1. The system displays a preview of the output video once generation is complete.

2. The user plays the output video to verify that it remains visually similar to the original and is still usable.

3.4.6 Use Case 6: Export Output Video

Primary Actor: User

Description: The user exports the final adversarial output video to the designated output folder.

Scenario:

1. The system displays an Export button once generation is complete.

2. The user clicks the button to save the output video.

3. The system saves the processed file to the output folder selected by the user.

3.5 User Interface Design

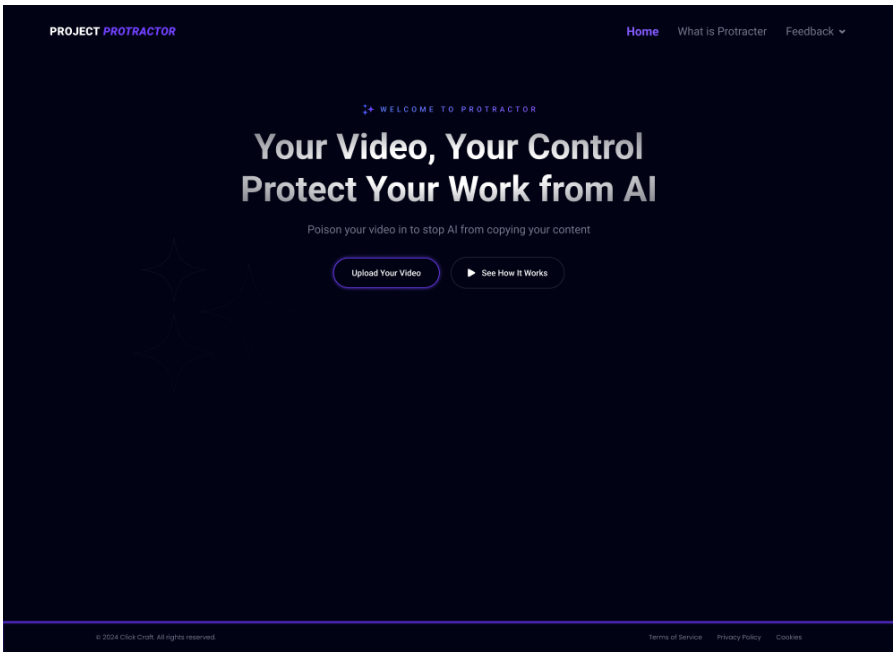

Figure 3.2: Mockup of the Home Page

The home page serves as the entry point of the application. It provides a brief introduction to Vid-Scratch and offers two main actions: uploading a video to begin the adversarial generation process and viewing an explanation of how the system works.

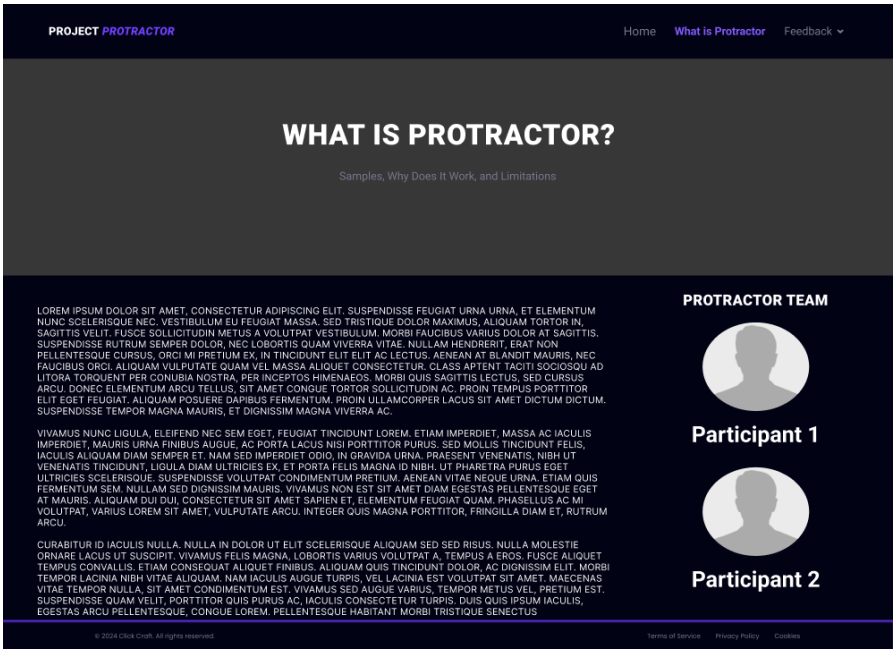

Figure 3.3: Mockup of the System Overview Page

The system overview page explains the purpose of Vid-Scratch, how adversarial video generation works, and the limitations that users should understand before using the application.

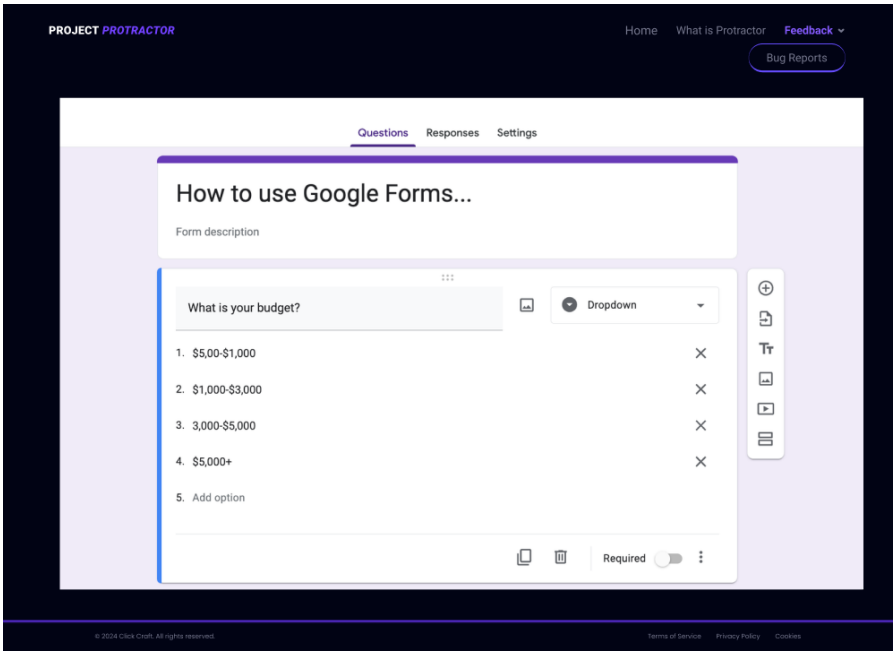

Figure 3.4: Mockup of the Feedback Page

The feedback page allows users to submit comments, suggestions, or bug reports about the application. This supports continuous improvement of the system based on user experience.

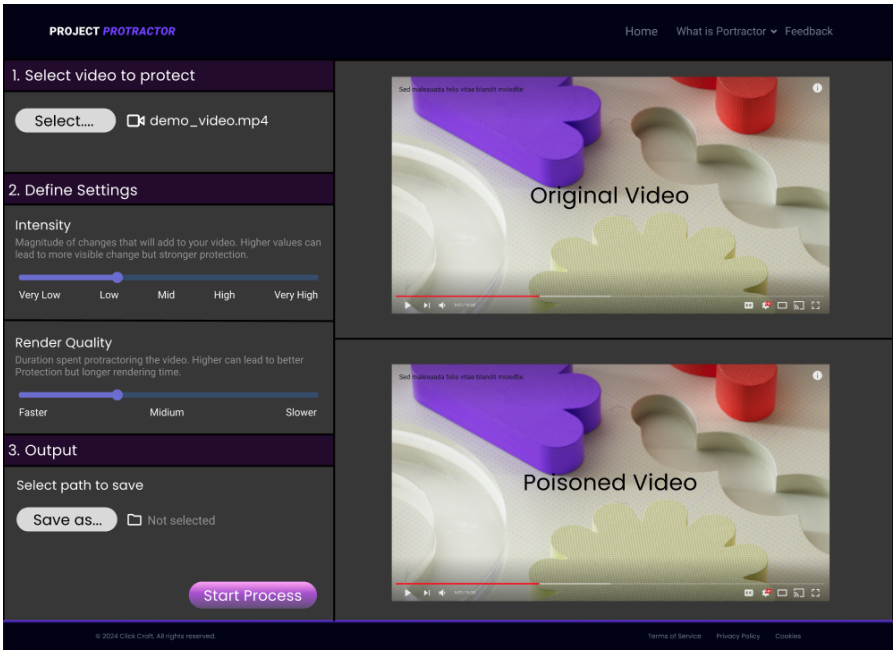

Figure 3.5: Mockup of the Main Process Page

The main process page supports the core workflow of the system. On this page, the user selects an input video, adjusts generation parameters such as perturbation intensity and render quality, selects an output folder, and starts the adversarial generation pipeline. The page also provides progress updates during processing and displays both the original and generated videos side by side for comparison.

Chapter 4

Software Architecture Design

4.1 Domain Model

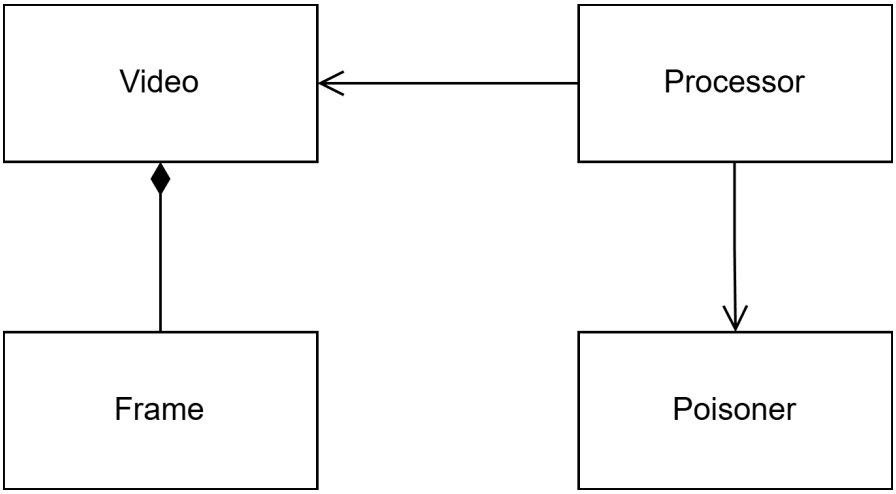

Figure 4.1: Domain Diagram

The domain model of Vid-Scratch describes the main entities involved in the adversarial video generation process. It shows how the system handles a video, extracts its frames, applies adversarial perturbations, and produces a protected output video.

4.1.1 Processor

The *Processor* acts as the main controller of the workflow. It receives the input video from the user, coordinates the frame extraction and poisoning process, and produces the final output video.

4.1.2 Poisoner

The *Poisoner* is responsible for applying adversarial perturbations to the extracted video frames. It receives individual frames or batches of frames from the Processor and modifies them according to the selected generation parameters.

4.1.3 Video

The *Video* represents the input and output video data in the system. It stores the essential information of a video, such as file path, duration, resolution, frame rate, and associated frames.

4.1.4 Frame

The *Frame* represents an individual image extracted from a video. It contains the frame data and related information required for adversarial modification before being passed back to the Processor for video reconstruction.

4.2 Design Class Diagram

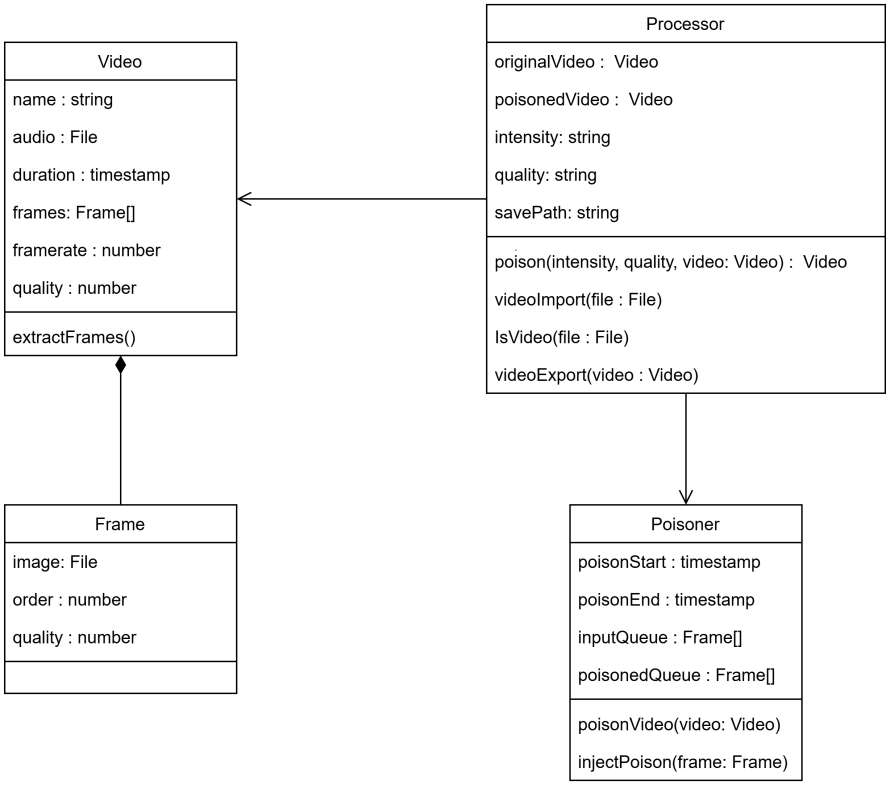

Figure 4.2: Class Diagram

The class diagram expands on the domain model by showing the attributes and methods of each class. The Processor class holds references to both the original and output video objects along with the generation parameters, and provides methods for importing, validating, and exporting video files. The Poisoner class manages the adversarial generation queue and provides methods for applying perturbations to individual frames. The Video class stores video metadata and its associated frames, while the Frame class holds the image data and ordering information for each extracted frame.

4.3 Sequence Diagram

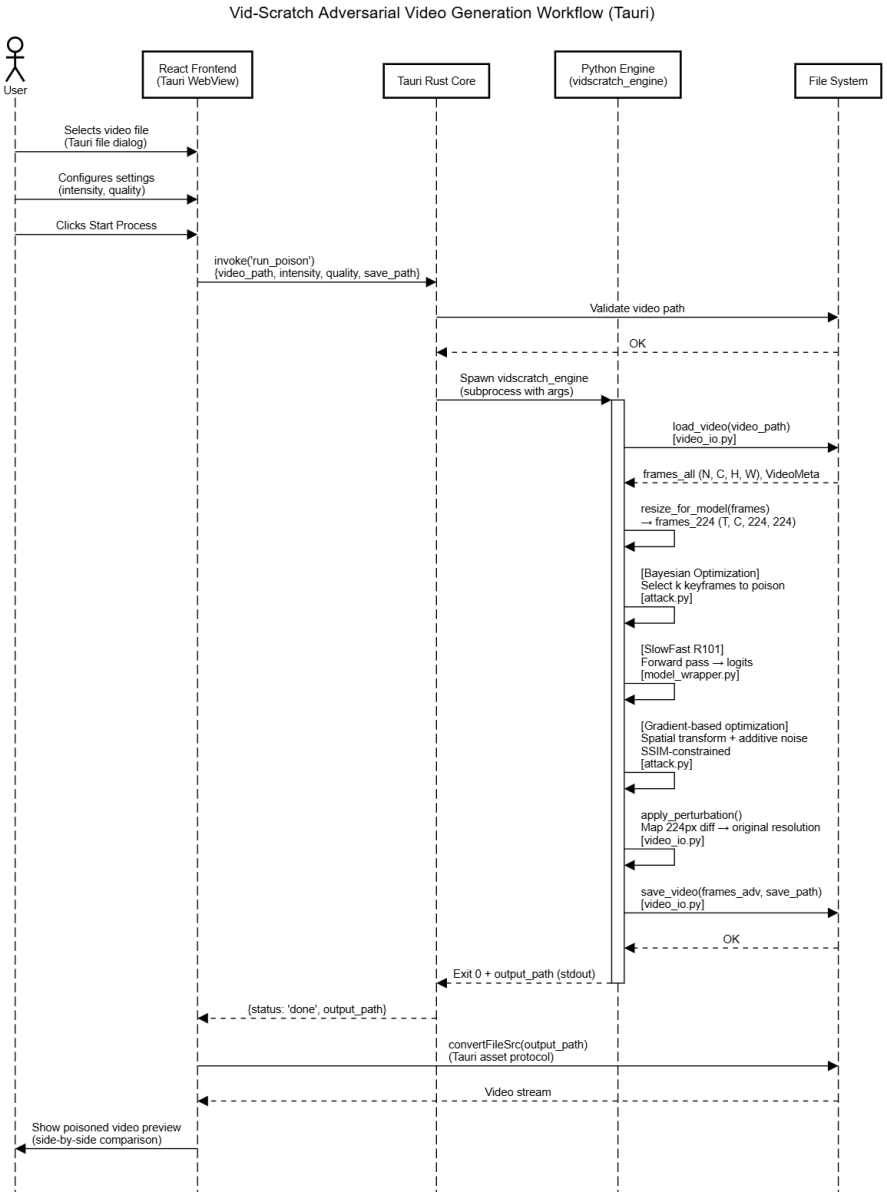

Figure 4.3: Sequence Diagram

The sequence diagram illustrates the interaction between system components during the adversarial video generation workflow. The user selects a video file and configures generation settings through the React

frontend embedded in the Tauri WebView. Once the user starts the process, the frontend invokes a Tauri Rust command, passing the video path, intensity, quality, and output path as arguments. The Rust core validates the input path and spawns the compiled Python engine as a subprocess. The Python engine then handles the processing pipeline in four stages: loading all frames from the input video at original resolution, selecting keyframes for perturbation using Bayesian Optimization, generating and applying adversarial perturbations through spatial transformation and gradient-based noise optimization constrained by SSIM, and saving the output video back to the file system. Once the engine exits successfully, the Rust core returns the output path to the frontend, which loads and displays the poisoned video alongside the original for the user to preview.

Chapter 5

AI Component

5.1 Business Context & AI Integration

Objective:

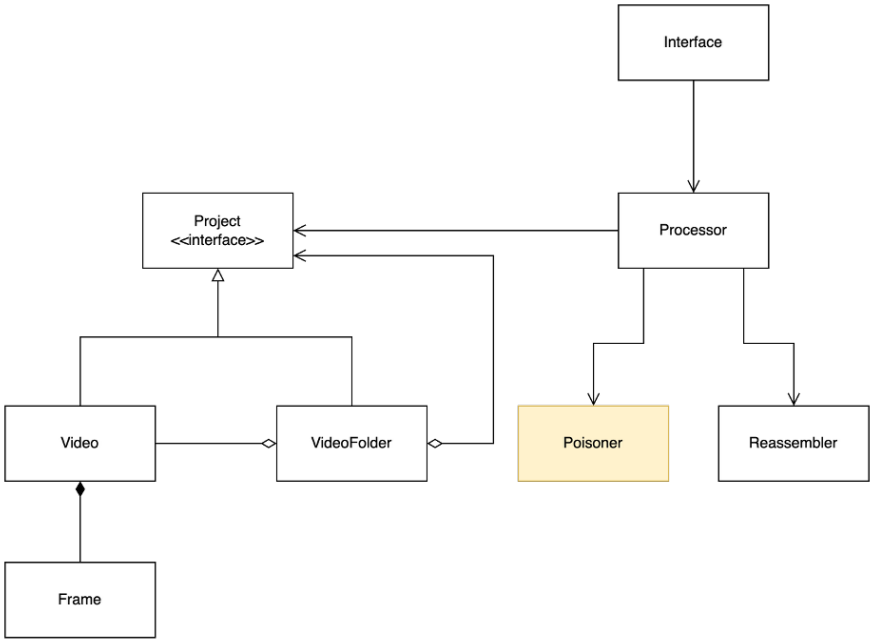

Figure 5.1: System's Domain Diagram

The domain diagram above illustrates the overall system structure of Vid-Scratch. The *Poisoner* component, highlighted in the diagram, represents the part of the system where AI is integrated to generate adversarial perturbations and apply them to the frames of the input video.

AI is suitable for this task because generating effective adversarial perturbations requires understanding how a model processes and clas-

sifies video input, particularly the feature representations and decision boundaries on which the model relies. Rather than applying random noise, the adversarial generation process uses gradient-based optimization to produce perturbations that are specifically designed to affect the model's predictions while remaining imperceptible to human viewers.

By applying perturbations that exploit the model's internal feature representations, the system aims to cause SlowFast R101 to incorrectly classify the action shown in the video at inference time, or to learn distorted patterns when trained on poisoned data. At the same time, the visual appearance of the output video remains close to the original, allowing the protected content to remain usable for human viewers.

5.2 Goal Hierarchy

Organization Goals

1. **Goal:** Develop an accessible application for generating adversarial video inputs targeting SlowFast R101 within the defined project scope.
   Measured by: Successful delivery of a working application that supports the generation of adversarial video outputs for both inference-time and training-time evaluation scenarios.
   Data collection: System functionality testing and academic evaluation by project supervisors.
   Operationalization: Verify that the application can generate adversarial video outputs and that the produced results are consistent with the expected behavior observed in both evaluation scenarios.

System Goals

1. **Goal:** Generate adversarial video outputs that influence SlowFast R101 model behavior while preserving visual similarity to the original video.
   Measured by: Attack success rate during inference-time evaluation, performance degradation during training-time evaluation, and percep-

tual similarity measurements such as SSIM.
Data collection: Model prediction logs, classification accuracy records, and perceptual similarity measurements on processed video samples.
Operationalization: Evaluate whether adversarially perturbed videos cause incorrect action classification by SlowFast R101, and whether poisoned training data reduces model accuracy, while confirming that SSIM remains above an acceptable threshold.

User Goals

1. **Goal:** Allow users to generate adversarial video variants through a simple and accessible workflow without requiring deep technical knowledge.
   Measured by: User satisfaction ratings and feedback on the usability and clarity of the application.
   Data collection: Post-use feedback forms and usability observations during system testing.
   Operationalization: Collect and analyze user feedback to assess whether the workflow is clear, easy to use, and produces outputs that meet expectations in terms of visual similarity.

AI Model Goals

1. **Goal:** Generate adversarial perturbations that are effective against SlowFast R101 by exploiting both the spatial and temporal structure of the input video.
   Measured by: Attack success rate during inference-time evaluation and accuracy reduction during training-time evaluation, compared with a baseline of unperturbed inputs.
   Data collection: SlowFast R101 prediction results on adversarial and clean video inputs, together with training accuracy curves on poisoned and clean datasets.
   Operationalization: Compare model predictions and training out-

comes between clean and adversarial conditions to quantify the impact of the generated perturbations on SlowFast R101 performance.

5.3 Task Requirements Analysis Using AI Canvas

5.3.1 AI Task Requirements

Requirements (REQ)

- REQ-1: The AI must generate adversarial perturbations that cause a pre-trained SlowFast R101 model to incorrectly classify the action shown in the input video at inference time.
- REQ-2: The AI must generate perturbed video samples that reduce the classification performance of a SlowFast R101 model trained on the perturbed dataset.
- REQ-3: The generated perturbations must preserve visual similarity to the original video as measured by perceptual similarity metrics such as SSIM.
- REQ-4: The AI must identify important frames for perturbation in order to improve attack effectiveness while keeping the number of modified frames minimal.

Specifications (SPEC)

- SPEC-1: The system implements the DeepSAVA pipeline, combining Bayesian Optimization for frame selection, spatial transformation via a learned flow field, and gradient-based additive noise optimization.
- SPEC-2: The perturbation strength ε and SSIM budget are configurable by the user through the application interface.
- SPEC-3: Perturbations are applied to a 40-frame representation of the input video sampled uniformly from the full clip.
- SPEC-4: For the training-time attack scenario, Error-Minimizing Noise is applied to non-attacked frames.

Environment (ENV)

- ENV-1: The system runs in a Python environment with PyTorch as the deep learning backend.

- ENV-2: Input data consists of MP4 video files of up to one minute in length.

- ENV-3: GPU acceleration is recommended for efficient adversarial generation, particularly for the optimization steps in the DeepSAVA pipeline.

- ENV-4: The backend runs as a compiled Python executable spawned by the Tauri Rust core, and is accessed through a React-based frontend embedded in the Tauri desktop application.

5.3.2 AI Canvas Development

Canvas Element	Description
Input Data	MP4 video files of up to one minute in length, uploaded by the user through the application interface.
Expected Output	Adversarially perturbed video files that cause SlowFast R101 to incorrectly classify the action at inference time, or perturbed video samples that reduce model performance when used as training data, while remaining visually similar to the original.
Tools/Frameworks	Python, PyTorch, OpenCV, and Tauri. The adversarial generation pipeline is based on DeepSAVA, incorporating Bayesian Optimization, spatial transformation, and gradient-based noise optimization, with SlowFast R101 pretrained on Kinetics-400 as the target model.
Success Criteria	Attack success against SlowFast R101 is observed during inference-time evaluation; training on perturbed data results in measurable performance reduction compared with clean data; and SSIM between the original and output video remains above an acceptable threshold.

5.4 User Experience Design with AI

5.4.1 Interaction Style

Interaction Style: Process and Display

The user interacts with the AI component indirectly through the application interface. The user selects an input video, configures generation parameters, and starts the processing pipeline. The AI then generates adversarial perturbations in the background and returns the output video for the user to preview and export. The system displays both the original and

output videos side by side so that the user can verify that visual similarity has been preserved.

5.4.2 User Feedback Mechanism

Users are encouraged to provide feedback to help improve the system. The application offers a dedicated Feedback page with the following options:

- **Comments and Suggestions** — Share ideas for improving the adversarial generation pipeline or the application interface.
- **Bug Reports** — Report technical issues or unexpected behavior during video processing.
- **Email Us** — Contact the development team directly for additional support.

5.4.3 AI Contribution to System Intelligence

Vid-Scratch integrates AI to perform gradient-based adversarial video generation targeting SlowFast R101. This approach provides capabilities that would be difficult to achieve through non-AI methods alone.

Without AI Integration

- Manual frame editing or fixed noise filters that do not account for how the model processes video input.
- Static perturbations that may be less effective against temporal aggregation in video models such as SlowFast R101.

With AI Integration

- Gradient-based optimization that targets the feature representations used by SlowFast R101.

- Bayesian Optimization for identifying the most influential frames to perturb, improving perturbation effectiveness while keeping modifications minimal.
- Spatial transformation combined with additive noise to exploit both the spatial and temporal structure of the video.
- Preservation of visual similarity between the original and output video through SSIM-constrained optimization.

Chapter 6

Software Development

6.1 Software Development Methodology

This project follows Extreme Programming (XP), an agile software development methodology that emphasizes short development cycles, continuous feedback, and close collaboration between team members and stakeholders. XP is suitable for this project because the team is small, the development timeline is fixed, and the technical requirements involve frequent experimentation and iteration, particularly in the adversarial generation pipeline.

The following characteristics of this project align with XP practices:

- **Small team:** The project is developed by two members, which supports close communication and a shared understanding of all components without requiring heavy coordination overhead.

- **Academic oversight:** Three supervisors provide consistent review and feedback throughout the development process, serving a role similar to the customer representative in XP.

- **Fixed timeline:** The project has a defined development period of approximately four months, which encourages prioritization and incremental delivery of working features.

- **Single codebase:** The entire system is maintained in a single shared repository, supporting continuous integration and consistent versioning across the frontend and backend components.

- **Continuous testing:** Adversarial generation results and system behavior are evaluated regularly throughout development to verify that the pipeline produces the expected outputs.

6.2 Technology Stack

Vid-Scratch is built using a combination of technologies across the desktop frontend and the adversarial generation backend.

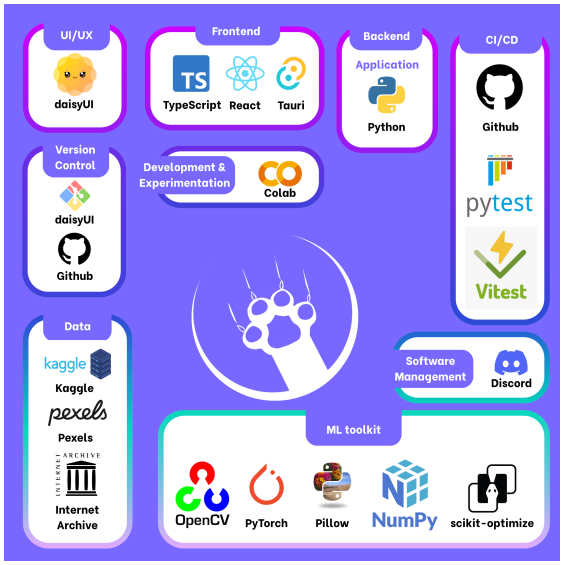

Figure 6.1: Technology stack of Vid-Scratch

6.2.1 Frontend

The desktop application is built with **Tauri**, a framework that packages a web-based user interface inside a native desktop shell. The user interface is written in **React** using **TypeScript** for static typing and **DaisyUI** as the component library for consistent styling. Tauri plugins, including the dialog and filesystem plugins, are used to handle native operations such as opening file dialogs and reading file paths from the local filesystem.

6.2.2 Backend and AI Pipeline

The adversarial generation pipeline is implemented in **Python** and exposed to the frontend through the command line. The pipeline uses **PyTorch** for all model inference and gradient-based optimization. The target model, SlowFast R101, is loaded via **PyTorchVideo** using `torch.hub`. [9] Video loading and frame preprocessing are handled with **OpenCV**, while **NumPy** and **Pillow** are used for frame-level operations. Bayesian Optimization for keyframe selection is implemented using **scikit-optimize**. [10]

6.2.3 Development and Experimentation Tools

Because a significant portion of the project involves model experimentation and attack evaluation, much of the research and prototyping work is conducted on **Google Colab** using GPU-accelerated runtimes. Version control is managed with **Git** and **GitHub**.

6.2.4 Data

Data used throughout the development and evaluation process was sourced from **Kaggle**, **Pexels**, and **Internet Archive**, covering a range of video datasets and media for the project's experiments and poison implementation.

6.3 Coding Standards

Vid-Scratch does not enforce a formal coding standard. The nature of this project is research-heavy: a large portion of the development involves experimenting with attack methods, tuning optimization parameters, and evaluating model behavior, much of which is carried out iteratively in notebooks and scripts rather than through a structured software engineering workflow. As a result, applying strict style enforcement or linting rules across all components was not practical within the project scope.

That said, the codebase follows general readability practices where possible. Python code is written with type hints and docstrings in key modules such as `model_wrapper.py` and `video_io.py` to improve clarity and maintainability. TypeScript is used throughout the frontend to provide static type checking. Function and variable names are chosen to reflect their purpose clearly, and related logic is grouped into separate modules to keep each file focused on a single responsibility.

6.4 Progress Tracking Report

Due to the research-oriented nature of this project, development progress does not follow a conventional sprint or task-completion model. A significant portion of the work involves exploring attack methods, evaluating experimental results, and iterating on the generation pipeline based on findings, much of which takes place in Google Colab notebooks and local experiment files rather than through commit-tracked feature development.

As a result, the GitHub contribution graph does not fully reflect the actual effort invested in the project. Experimental work, parameter tuning, and result analysis carried out outside the repository are not captured in commit history. The progress of the project is therefore better described through the research decisions and evaluation results documented in the subsequent chapters of this report.

Chapter 7

Deliverable

7.1 Software Solution

Vid-Scratch is delivered as a desktop application built with Tauri, providing a self-contained adversarial video poisoning pipeline that runs entirely on the user's local machine. The source code is available at the project GitHub repository.

This section describes the full poisoning pipeline from video input to adversarial video output, followed by screenshots of the application user interface.

7.1.1 Poisoning Pipeline

The pipeline transforms an input video into an adversarially perturbed video through six sequential stages, as illustrated below.

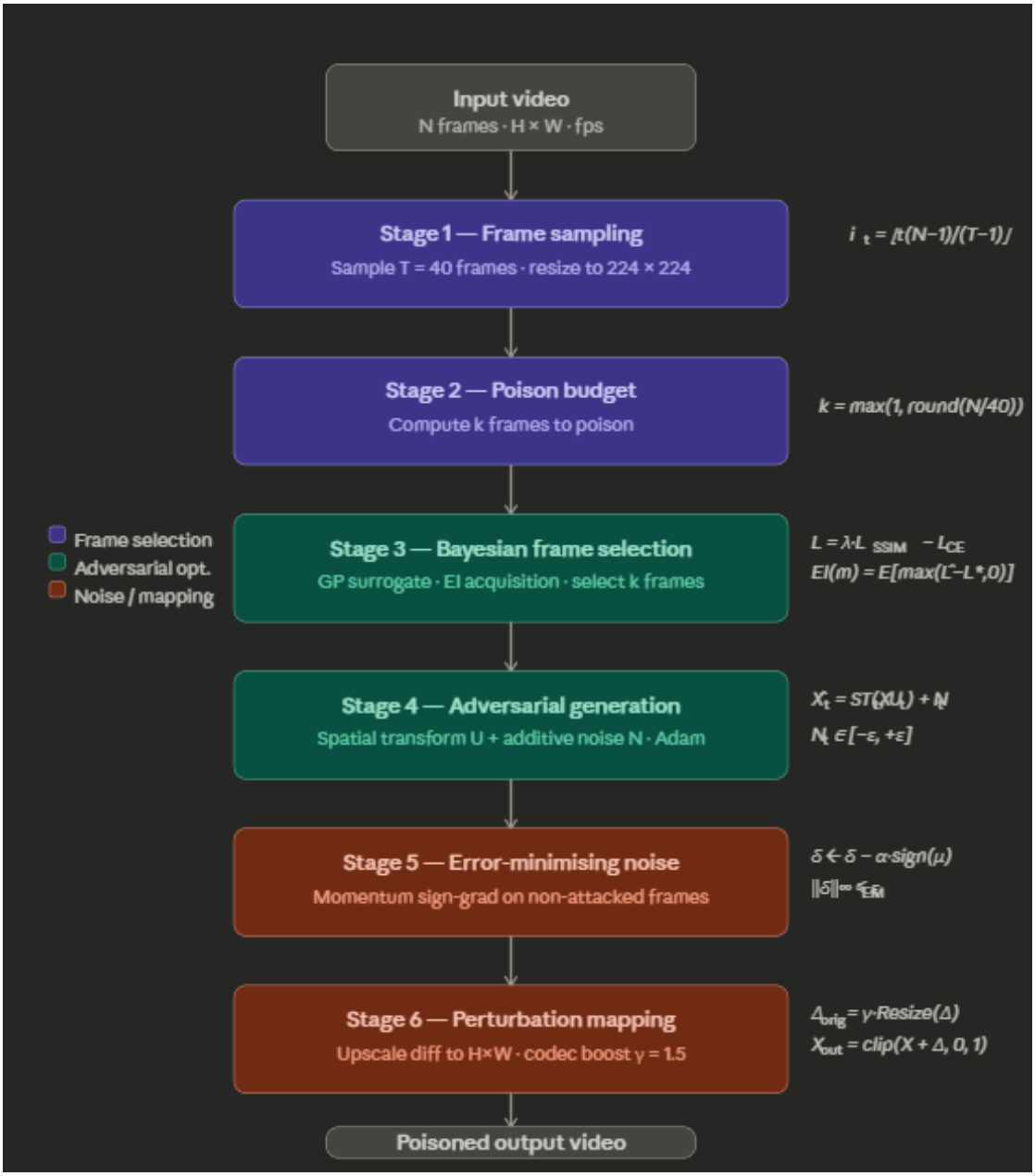

Figure 7.1: Overview of the Vid-Scratch adversarial poisoning pipeline.

Stage 1: Video Loading and Frame Sampling

The input video is decoded frame by frame using OpenCV. All N frames are loaded at their original resolution $(H_{\text{orig}} \times W_{\text{orig}})$. A model

window of $T = 40$ uniformly sampled frames is then selected from the full sequence using linear indexing:

$$i_t = \left\lfloor \frac{t \cdot (N-1)}{T-1} \right\rfloor, \quad t = 0, 1, \ldots, T-1 \tag{7.1}$$

where i_t is the index of the t-th sampled frame in the original sequence. The sampled frames are then resized to 224×224 pixels for model input.

Stage 2: Poison Frame Budget

The number of frames to poison, k, is computed automatically from the total frame count:

$$k = \max\left(1,\ \text{round}\left(\frac{N}{40}\right)\right) \tag{7.2}$$

For a short video of $N < 40$ frames, $k = \max(1, \lfloor 0.25N \rfloor)$ is used instead to ensure at least one frame is poisoned. The budget k controls how many frames in the 40-frame model window will receive adversarial perturbations.

Stage 3: Bayesian Frame Selection (Algorithm 1)

Rather than perturbing all T frames equally, Vid-Scratch selects the k *most vulnerable* frames using Bayesian Optimisation (BO) with a Gaussian Process (GP) surrogate model.

A candidate binary mask $\mathbf{m} \in \{0,1\}^T$ assigns value 1 to k selected frames and 0 to the rest. For each candidate mask, a proxy adversarial loss is evaluated:

$$\mathscr{L}_{\text{proxy}}(\mathbf{m}) = \lambda\, \mathscr{L}_{\text{SSIM}}(\hat{X}, X) - \mathscr{L}_{\text{CE}}(f(\hat{X}), y) \tag{7.3}$$

where X is the clean model-window tensor, $\hat{X}$ is its perturbed version under mask $\mathbf{m}$, $f(\cdot)$ is the SlowFast R101 model, y is the ground-truth label, $\mathscr{L}_{\text{CE}}$ is the cross-entropy loss, and $\mathscr{L}_{\text{SSIM}}$ is the SSIM-based perceptual similarity loss.

The GP models the mapping from mask to proxy loss. At each BO iteration, the next candidate is chosen by maximising the Expected Improvement (EI) acquisition function:

$$\mathrm{EI}(\mathbf{m}) = \mathbb{E}\Big[\max\Big(\hat{\mathscr{L}}(\mathbf{m}) - \mathscr{L}^* - \xi,\, 0\Big)\Big] \tag{7.4}$$

where $\mathscr{L}^*$ is the best observed proxy loss so far and $\xi > 0$ is an exploration parameter. After N_{BO} iterations (default: 50), the mask with the highest proxy loss is selected.

Stage 4: Adversarial Generation (Algorithm 2)

Given the selected mask $\mathbf{m}^*$, the adversarial perturbation is optimised jointly over a spatial flow field $U \in \mathbb{R}^{T\times 2\times H\times W}$ and an additive noise tensor $N \in \mathbb{R}^{T\times C\times H\times W}$.

The perturbed frame sequence is constructed as:

$$\hat{X}_t = \begin{cases} (\mathscr{S}\mathscr{T}(X_t, U_t) + N_t)_{[0,1]} & \text{if } m_t = 1 \\ X_t & \text{if } m_t = 0 \end{cases} \tag{7.5}$$

where $\mathscr{S}\mathscr{T}(\cdot, U_t)$ is a differentiable spatial transformer that warps frame X_t according to flow field U_t, and $(\cdot)_{[0,1]}$ denotes pixel-level clamping to $[0,1]$.

The full objective minimised by Adam is:

$$\mathscr{L} = \lambda\, \mathscr{L}_{\mathrm{SSIM}}\big(\hat{X}_{\mathscr{S}},\, X_{\mathscr{S}}\big) - \mathscr{L}_{\mathrm{CE}}\big(f(\hat{X}),\, y\big) \tag{7.6}$$

where $\mathscr{S} = \{t : m_t = 1\}$ is the set of poisoned frames. Minimising $\mathscr{L}$ simultaneously encourages high perceptual similarity on poisoned frames (via the SSIM term) and maximises the classification loss so that the model is more likely to misclassify (via the adversarial term).

At each optimisation step, the noise is clamped:

$$N_t \leftarrow \mathrm{clip}(N_t,\, -\varepsilon,\, +\varepsilon) \tag{7.7}$$

where ε is the noise budget (default: $\varepsilon = 0.03$). Optimisation also terminates early if the SSIM value on poisoned frames drops below the SSIM budget threshold $1 - \delta_{\text{SSIM}}$ (default: $\delta_{\text{SSIM}} = 0.03$).

Stage 5: Error-Minimising Noise (Training-Time Layer)

For the training-time attack scenario, a second noise layer is applied to the *non-attacked* frames using the Error-Minimising (EM) noise strategy. This layer is designed to reduce the usefulness of the poisoned video as a training sample by minimising the classification loss on the remaining frames.

The EM noise δ_t is updated iteratively using momentum-based gradient descent with sign gradient:

$$\mu_{s+1} = \beta\, \mu_s + \frac{\nabla_\delta \mathscr{L}_{\text{CE}}(f(X_t + \delta_t), y)}{\|\nabla_\delta \mathscr{L}_{\text{CE}}\|_1 + \varepsilon_0} \tag{7.8}$$

$$\delta_t \leftarrow \delta_t - \alpha \cdot \text{sign}(\mu_{s+1}) \tag{7.9}$$

where $\beta = 0.9$ is the momentum coefficient, α is the step size, and ε_0 is a small stabiliser. The noise is constrained by $\|\delta_t\|_\infty \leq \varepsilon_{\text{EM}}$ (default: $\varepsilon_{\text{EM}} = 0.002$), keeping the base layer imperceptible to human viewers.

Stage 6: Perturbation Mapping and Output

The adversarial perturbation is computed at the 224-pixel model resolution. To produce the output video at the original resolution, the difference $\Delta_t = \hat{X}_t^{(224)} - X_t^{(224)}$ is upscaled back:

$$\Delta_t^{(\text{orig})} = \gamma \cdot \text{Resize}\big(\Delta_t,\, H_{\text{orig}} \times W_{\text{orig}}\big) \tag{7.10}$$

where γ is a codec boost factor (default: $\gamma = 1.5$) that compensates for perturbation attenuation caused by video compression. The final output frame is:

$$X_t^{(\text{out})} = \text{clip}\Big(X_t^{(\text{orig})} + \Delta_t^{(\text{orig})},\, 0,\, 1\Big) \tag{7.11}$$

All N original frames are preserved in the output video, with only the k poisoned frames carrying the adversarial perturbation. The output video is saved at the original resolution, frame rate, and with audio from the source file remuxed without re-encoding.

7.1.2 User Interface

The Vid-Scratch desktop application provides a three-step workflow: select a video, configure settings, and run the poisoning pipeline. The following figures illustrate the main screens of the application.

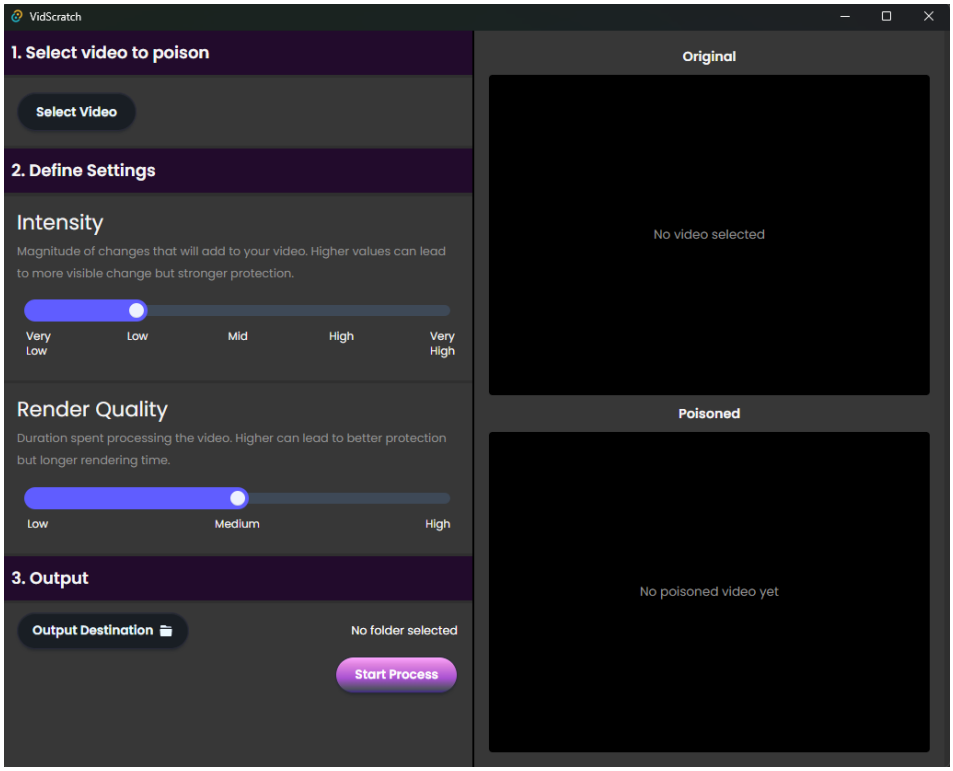

Figure 7.2: Initial state of the application. The left panel shows the three-step workflow. The right panel shows two video preview areas (Original and Poisoned), both empty before a video is selected.

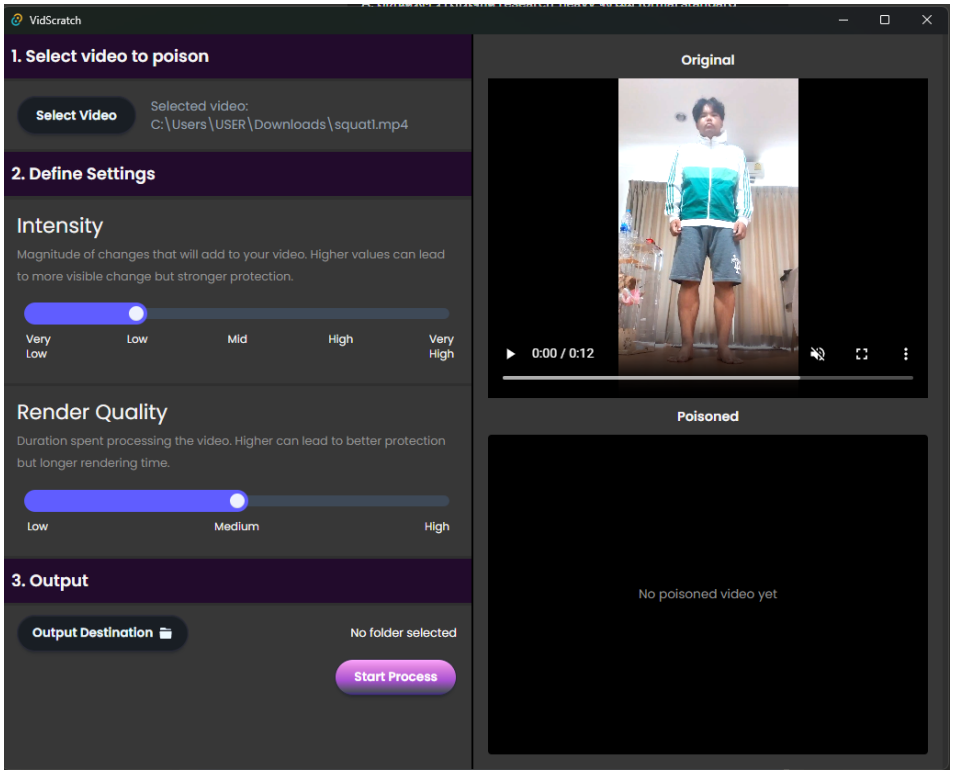

Figure 7.3: After the user selects an input video via the native file dialog, the original video is loaded into the preview panel and the selected file path is displayed. The Poisoned panel remains empty until the pipeline has been executed.

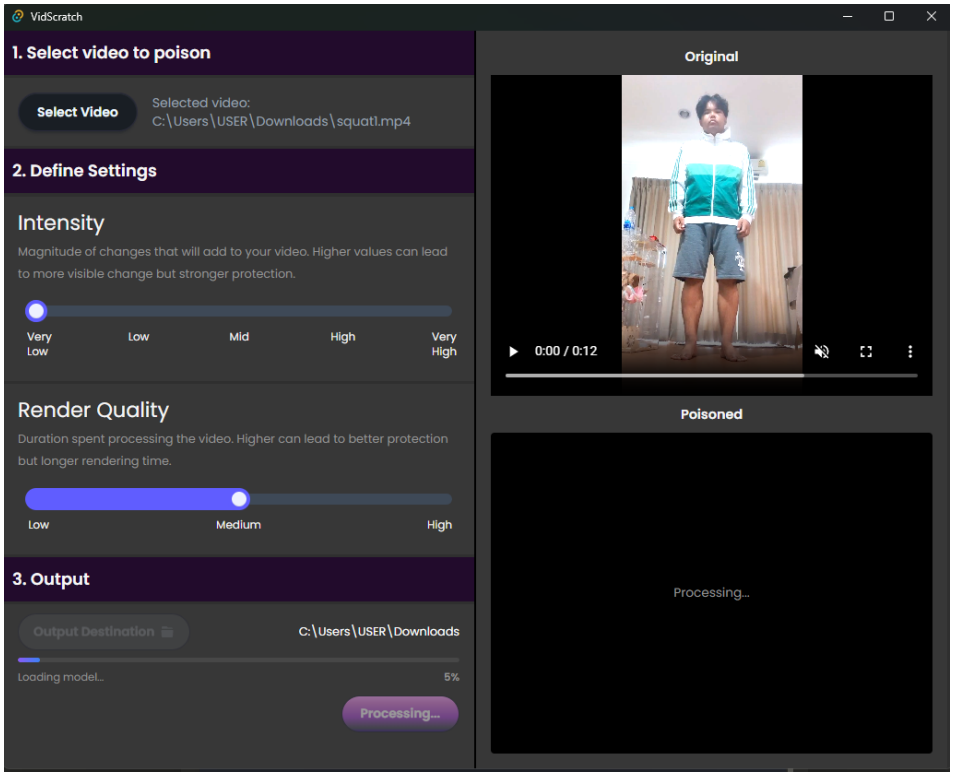

Figure 7.4: Processing state. After the user clicks *Start Process*, the button changes to *Processing...* and a progress bar appears below the output destination showing the current stage (e.g., "Loading model... 5%"). The Poisoned panel displays a *Processing...* placeholder while the pipeline runs.

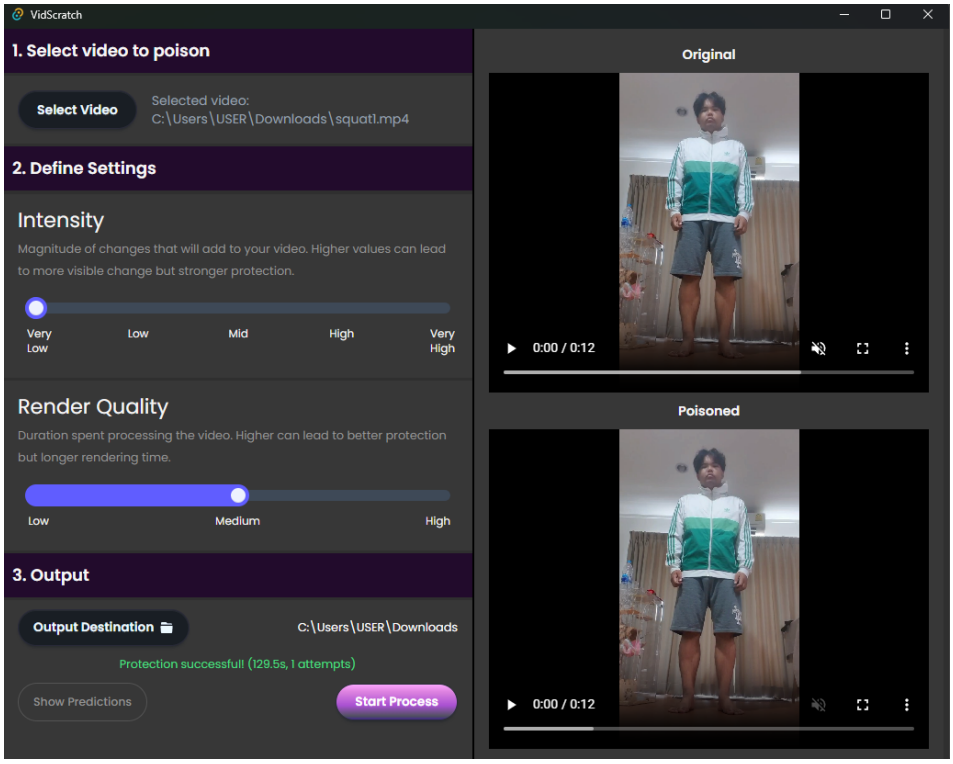

Figure 7.5: Completed state. Both the original and poisoned videos are shown side by side in the preview panels. The output section displays a *Protection successful!* message along with the total processing time and number of retry attempts. A *Show Predictions* button appears to reveal the model prediction panel.

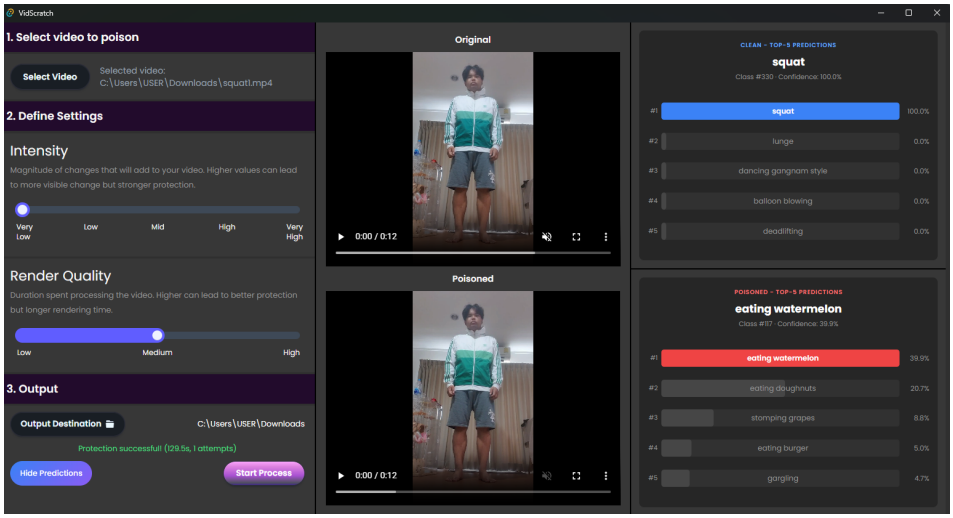

Figure 7.6: Prediction panel expanded. The right side shows two prediction blocks: *Clean — Top-5 Predictions* displays the model's confidence scores on the original video (e.g., *squat* at 100.0%), and *Poisoned — Top-5 Predictions* displays the model's predictions on the poisoned video (e.g., *eating watermelon* at 39.9%), confirming that the attack successfully caused the model to misclassify the action.

7.2 Test Report

The testing of Vid-Scratch is divided into three parts: an inference-time benchmark on real Kinetics-400 videos, a training-time poisoning experiment, and a unit test suite covering the core pipeline modules.

7.2.1 Inference-Time Attack Benchmark

The inference-time attack was evaluated on 10 video clips sampled from the Kinetics-400 dataset using the default pipeline configuration [11] ($\varepsilon = 0.03$, $\delta_{\text{SSIM}} = 0.03$, $N_{\text{BO}} = 50$, $T_{\max} = 150$). Each video was processed by the full VidScratch pipeline and the output was verified by reloading the saved file and running SlowFast R101 inference again to confirm that the attack holds after codec compression.

The results are summarised in Table 7.1.

Table 7.1: Inference-time attack results on 10 Kinetics-400 videos. Orig = original predicted class index, Adv = adversarial predicted class index, SSIM = structural similarity between original and poisoned video.

#	Video	Orig	Adv	SSIM	Time (s)
1	01R6cmvFbZI	108	353	0.9441	47.6
2	-2VKVjgNuE0	305	105	0.9555	35.0
3	–SrV9bGzRA	310	160	0.9548	51.0
4	0G2A0x2-vf4	117	114	0.9497	35.1
5	-ayNZ5bs7EQ	289	256	0.9237	34.9
6	-9GW_m6MQd0	103	1	0.9268	45.1
7	-7mJ12n-xyM	335	304	0.9726	42.1
8	-3zcsBnDVzU	145	363	0.9542	41.4
9	0fjsXnccCu0	318	134	0.9768	46.4
10	-2FbowxhzfQ	301	222	0.9428	41.6
Attack success rate					**10/10 (100%)**
Verified success rate					**10/10 (100%)**
Average SSIM					**0.9501**
Average time					**42.0 s/video**

All 10 videos were successfully misclassified both in memory and after saving and reloading the output file, giving a verified attack success rate of 100%. The average SSIM across all poisoned videos was 0.9501, indicating that the perturbations remain visually close to the originals. Average processing time was 42.0 seconds per video on CPU.

7.2.2 Training-Time Poisoning Experiment

To evaluate the training-time attack scenario, a small classification model was trained under three data conditions and evaluated on a held-out clean test set at every epoch over 80 epochs. The three conditions were:

1. **Clean model:** trained on 100% clean videos.

2. **Poison model:** trained on 100% poisoned videos.
3. **Mixed model:** trained on 50% poisoned and 50% clean videos.

The dataset consisted of two action classes (lunges and squats) drawn from a UCF-101 subset [12], with 60 videos per class and a 50/60 train split. The target model was a lightweight two-class classifier fine-tuned from SlowFast R101 features. Training used a batch size of 4, a learning rate of 0.0005, and ran for 80 epochs.

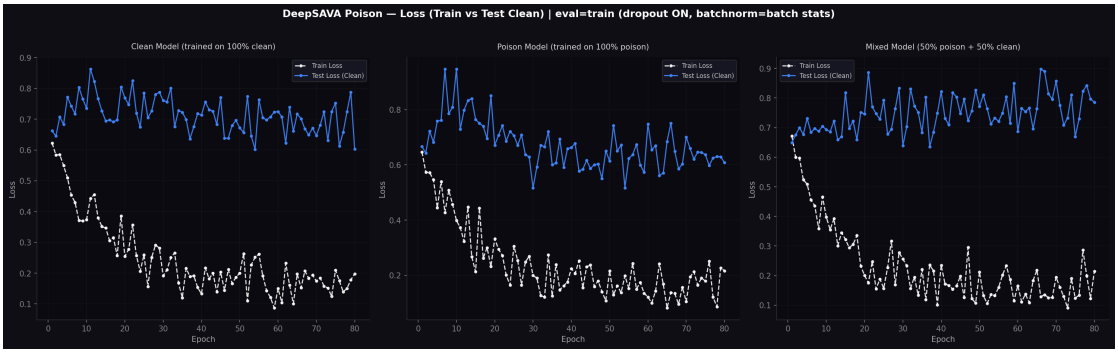

Figure 7.7: Training loss (white) and test loss on clean data (blue) across 80 epochs for three training conditions. Left: clean model. Centre: poison model. Right: mixed model (50% poison + 50% clean).

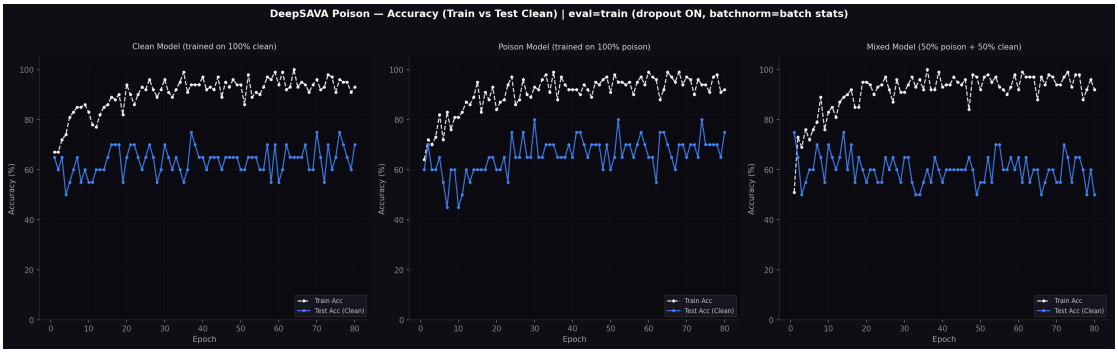

Figure 7.8: Training accuracy (white) and test accuracy on clean data (blue) across 80 epochs for three training conditions. Left: clean model. Centre: poison model. Right: mixed model (50% poison + 50% clean).

The results show that all three models achieve high training accuracy (above 90%) by the end of training, which is expected as each model

is able to overfit its own training distribution. The key observation is in the test loss on clean data. The clean model shows a clear downward trend in test loss, reflecting genuine generalisation to unseen clean samples. The mixed model, by contrast, shows a slightly increasing trend in test loss on clean data despite the training loss continuing to decrease, which is consistent with the poisoned samples introducing conflicting gradient signals that partially disrupt clean generalisation.

These results should be interpreted with caution for two reasons. First, the dataset is small (approximately 60 videos per class), which causes high variance in the loss curves and makes it difficult to draw statistically strong conclusions. Second, the poisoning was applied using a SlowFast R101 model trained on all 400 Kinetics classes, whereas the downstream classifier was trained on only two classes, which reduces the alignment between the adversarial perturbation and the training objective. Despite these limitations, the mixed model's test loss trend suggests that poisoned data can introduce measurable interference in clean generalisation even at low noise levels, which is consistent with the expected behaviour of the error-minimising noise component.

7.2.3 Unit Testing

Unit tests covering the core pipeline modules are maintained in the project GitHub repository. The test suite verifies the correctness of the video loading and frame sampling functions in `video_io.py`, the SlowFast R101 wrapper in `model_wrapper.py`, the Bayesian frame selector and adversarial generator in `vidscratch.py`, and the perturbation mapping and output saving functions.

All unit tests pass on the current codebase. No continuous integration or deployment pipeline is configured for this project, as the system is distributed as a local desktop application and does not involve a hosted backend service.

Chapter 8

Conclusion and Discussion

8.1 Challenges and Solutions

8.1.1 Finding an Effective Low-Noise Poisoning Method

The most significant challenge in this project was identifying an adversarial perturbation method that could cause a video action recognition model to misclassify while keeping the noise imperceptibly small. Most published research in this area focuses on high-visibility perturbations or physical-world attacks, such as adversarial patches or printed eyeglass frames, which are not suitable for the use case of protecting user-uploaded video content.

The team surveyed multiple attack methods from the literature, including universal adversarial perturbations, black-box transfer attacks, and feature-space poisoning approaches, before settling on the sparse adversarial spatial-transform approach from Mu et al. (BMVC 2021). Even after selecting this method, adapting it to work reliably on SlowFast R101 required extensive experimentation with the noise budget, SSIM constraint, Bayesian Optimisation parameters, and the codec boost factor to compensate for compression attenuation.

8.1.2 GPU Compatibility with Bayesian Frame Selection

The Bayesian frame selection component runs a short inner optimisation loop for each candidate mask evaluation. On CPU this is manageable for a small number of candidates, but the full BO procedure with 50 iterations is computationally intensive. Testing on Apple Silicon MacBooks revealed that the MPS backend does not fully support the tensor operations used in the spatial transformer and the GP kernel

computations, causing the pipeline to fall back to CPU. A single BO pass on CPU without GPU acceleration was observed to take approximately one hour per video, which made iterative development impractical.

This was partially addressed by reducing the number of BO iterations during development and running full evaluations on CUDA-enabled hardware. The final pipeline defaults to CUDA when available and falls back to CPU otherwise, with a note to users that processing time on CPU may be significantly longer.

8.1.3 Dependency on a Large Pretrained Model

SlowFast R101 is a large model that must be downloaded from the PyTorchVideo hub on first use. This creates a dependency on an internet connection during the initial setup and adds approximately 200 MB to the application footprint. The model is cached after the first download, but users on slow connections or restricted networks may encounter difficulties. This was mitigated by adding a loading progress indicator in the application and providing clear error messages when the model cannot be fetched.

8.1.4 Background in AI and Data Poisoning

At the start of the project, both team members had limited prior experience with adversarial machine learning and video model architectures. To build the necessary background, the team participated in a series of AI workshop labs advised by the project supervisor, beginning in January 2025. Topics covered included binary image classification, the stAdv spatial-transform attack, text-to-video model training, and quantifiable metrics for evaluating model degradation under adversarial inputs. This foundational work was essential for understanding the limitations of existing approaches and making informed design decisions throughout the project.

8.2 Limitations

8.2.1 GPU Requirement for Practical Use

The full VidScratch pipeline with Bayesian Optimisation is not practical to run on CPU for most users. On a modern laptop without a CUDA-capable GPU, processing a single short video can take over an hour. This limits the accessibility of the application for users who do not have access to a discrete GPU, which contradicts the project's goal of providing an accessible tool for everyday users.

8.2.2 Limited Impact on Training-Time Attacks

The training-time poisoning experiment demonstrated that the perturbations produced by VidScratch have only a modest effect on a model trained on poisoned data. The adversarial noise is optimised to fool a 400-class SlowFast R101 model, but when the downstream model is trained on a small two-class dataset the gradient signal from the perturbation is not well aligned with the training objective. The result is a subtle increase in test loss rather than a clear degradation in model performance. For the training-time attack to be more effective, the perturbation would need to be generated with knowledge of the downstream training setup, which is not available in a practical user-facing tool.

8.2.3 Fixed Target Model

The current implementation targets SlowFast R101 exclusively. Users who want to test robustness against a different action recognition model, or who encounter a platform that uses a different architecture, cannot use VidScratch directly without modifying the codebase. The pipeline is designed to be extensible, but swapping the target model requires technical knowledge of the Python backend.

8.2.4 Short Video Constraint

The pipeline is designed for videos of up to one minute in length, which covers most short-form content but excludes longer recordings. Extending support to longer videos would require changes to the frame sampling and memory management logic, as processing all frames of a long video at full resolution would exceed the memory available on most consumer hardware.

8.3 Future Work

8.3.1 Support for Transformer-Based Video Models

The most natural extension of this project is to target more recent video understanding architectures, particularly transformer-based models such as VideoMAE, TimeSformer, and Video Swin Transformer. These models have largely replaced CNN-based architectures such as SlowFast on standard benchmarks. Generating adversarial perturbations that transfer across model families — or that are effective against attention-based temporal reasoning specifically — remains an open and technically interesting problem.

8.3.2 Improved Training-Time Effectiveness

Future work could explore poisoning strategies that are more directly aligned with the downstream training objective. One direction is to generate perturbations that target the feature representations learned by the model rather than its final classification output. Targeting intermediate feature layers tends to produce more transferable and training-disruptive perturbations, and may remain effective even when the downstream model differs from the attack model.

8.3.3 GPU-Efficient Frame Selection

Replacing the current Gaussian-Process-based Bayesian Optimisation with a lighter alternative, such as a gradient-based saliency score or

a simple frame-level loss ranking, could substantially reduce processing time without significantly reducing attack effectiveness. This would make the pipeline usable on CPU in a practical amount of time and improve accessibility for users without dedicated GPU hardware.

8.3.4 Broader Video Format Support

The current version supports MP4 input and output only. Future versions could extend support to additional formats such as AVI, MKV, and MOV, and could also support frame-rate-aware perturbation to handle high-frame-rate content more consistently.

8.4 Privacy and Data Usage

Vid-Scratch processes all video data locally on the user's machine. No video content, frame data, or personal information is transmitted to any external server at any point during the poisoning pipeline. The application does not collect usage data or telemetry. Adversarial outputs produced by the pipeline are saved to a user-specified local folder and are not shared or made accessible to any third party.

The project did not use any copyrighted material without permission during development. Videos used for evaluation were drawn from the publicly available Kinetics-400 and UCF-101 datasets under their respective research-use licenses. Any user-supplied video content is used solely for the purpose of generating the adversarial output requested by the user and is not retained by the application.

8.5 Conclusion

This project presented Vid-Scratch, a desktop application that generates adversarially perturbed video clips designed to cause a pre-trained action recognition model to misclassify the action shown in the input video, while preserving high visual similarity to the original content. The system targets SlowFast R101 pretrained on Kinetics-400 and implements

a two-stage pipeline consisting of Bayesian Optimisation for sparse frame selection followed by joint optimisation of a spatial transformation field and additive noise using Adam.

The inference-time attack achieved a verified success rate of 100% across 10 Kinetics-400 test videos with an average SSIM of 0.9501, demonstrating that the approach can reliably cause misclassification while keeping the perturbation imperceptible to casual inspection. The training-time experiment showed a modest but observable effect on clean test loss when poisoned samples were mixed into the training data, though the limited dataset size and the mismatch between the 400-class attack model and the 2-class training setup reduce the strength of this result.

Beyond the technical results, the project also produced an accessible desktop application that allows non-expert users to apply adversarial video poisoning without requiring knowledge of the underlying machine learning methods. In this respect, Vid-Scratch bridges adversarial machine learning research and a more practical application setting, and serves as a demonstration of how strong benchmark performance in action recognition does not necessarily imply robustness under carefully constructed adversarial inputs.

Several limitations remain, most notably the requirement for a CUDA-capable GPU for practical use and the limited effectiveness of the training-time attack under the current noise budget. These point to clear directions for future work, including support for transformer-based video models, GPU-efficient frame selection, and more targeted training-time poisoning strategies. With these improvements, a future version of the system could provide a more complete and practically useful tool for evaluating and demonstrating the adversarial vulnerability of video understanding models.

Reference

Bibliography

[1] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," in *IEEE Transactions on Image Processing*, vol. 13, no. 4, 2004, pp. 600–612.

[2] J. Carreira and A. Zisserman, "Quo vadis, action recognition? A new model and the Kinetics dataset," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 6299–6308.

[3] C. Feichtenhofer, H. Fan, J. Malik, and K. He, "SlowFast networks for video recognition," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 6202–6211.

[4] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *International Conference on Learning Representations (ICLR)*, 2015.

[5] R. Mu, W. Ruan, L. S. Marcolino, and Q. Ni, "Enhancing robustness in video recognition models: Sparse adversarial attacks and beyond," *Neural Networks*, 2023.

[6] C. Xiao, J.-Y. Zhu, B. Li, W. He, M. Liu, and D. Song, "Spatially transformed adversarial examples," in *International Conference on Learning Representations (ICLR)*, 2018.

[7] H.-S. Kim *et al.*, "Breaking temporal consistency: Generating video universal adversarial perturbations using image models," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[8] R. Mu, W. Ruan, L. S. Marcolino, and Q. Ni, "Sparse adversarial video attacks with spatial transformations," in *Proceedings of the British Machine Vision Conference (BMVC)*, 2021.

[9] H. Fan, Y. Li, B. Xiong, W.-Y. Lo, and C. Feichtenhofer, "PySlowFast," https://github.com/facebookresearch/SlowFast, 2020.

[10] F. Nogueira, "Bayesian optimization: Open source constrained global optimization tool for Python," https://github.com/bayesian-optimization/BayesianOptimization, 2014.

[11] W. Kay, J. Carreira, K. Simonyan, B. Zhang, C. Hillier, S. Vijayanarasimhan, F. Viola, T. Green, T. Back, P. Natsev, M. Suleyman, and A. Zisserman, "The Kinetics human action video dataset," *arXiv preprint arXiv:1705.06950*, 2017.

[12] K. Soomro, A. R. Zamir, and M. Shah, "UCF101: A dataset of 101 human actions classes from videos in the wild," *arXiv preprint arXiv:1212.0402*, 2012.

[13] Overleaf, "Learn LaTeX in 30 minutes," https://www.overleaf.com/learn/latex/Learn_LaTeX_in_30_minutes, 2023.

Appendix A

Appendix A: Example

<TIP: Put additional or supplementary information/data/figures in appendices. />

Appendix B

Appendix B: About LaTeX

LaTeX (stylized as LaTeX) is a software system for typesetting documents. LaTeX markup describes the content and layout of the document, as opposed to the formatted text found in WYSIWYG word processors like Google Docs, LibreOffice Writer, and Microsoft Word. The writer uses markup tagging conventions to define the general structure of a document, to stylize text throughout a document (such as bold and italics), and to add citations and cross-references.

LaTeX is widely used in academia for the communication and publication of scientific documents and technical note-taking in many fields, owing partially to its support for complex mathematical notation. It also has a prominent role in the preparation and publication of books and articles that contain complex multilingual materials, such as Arabic and Greek.

Overleaf has also provided a 30-minute guide on how you can get started on using LaTeX. [13]